

Зачет ОПД

Зачет ОПД

1. Операционные системы.....	3
2. История Unix/Linux.....	3
3. Современность.....	4
4. Ядро *NIX.....	4
5. Файловая система.....	5
6. Права доступа к файлам.....	5
7. Способы задания прав.....	6
8. Поток stdin(0), stdout(1), stderr(2).....	6
9. Интерпретатор команд.....	6
10. Перенаправление потоков std.....	6
11. Фильтры.....	7
12. Регулярные выражения.....	7
13. Команды.....	7
14. Аналоговые ЭВМ.....	8
15. Цифровые ЭВМ.....	9
16. Функциональные элементы ЭВМ.....	9
17. Первая ЭВМ: Калькулятор.....	11
18. Архитектура ЭВМ Гарвардская, фон Неймана.....	12
19. Структура БЭВМ-NG.....	12
20. Устройство управления:.....	13
21. Адресуемая память БЭВМ:.....	14
22. АЛУ, коммутатор, блок признаков результата:.....	14
23. Форматы команд:.....	14
24. Адресные команды.....	16
25. Безадресные команды.....	16
26. Представление чисел: фиксированная точка.....	17
27. Представление беззнаковых целых чисел.....	18
28. Представление знаковых целых чисел.....	18
29. Представление знаковых чисел: дополнительный код.....	18
30. Перенос, Переполнение.....	19
31. БЭВМ: представление чисел.....	20
32. Представление чисел с плавающей точкой.....	20
33. Представление логической информации.....	21
34. Представление символьной и текстовой информации.....	22

35. Символы: ASCII.....	22
36. Символы: ASCII (КОИ-7Н0), КОИ-7Н1 (РУС), КОИ7Н2 (Mix)....	22
37. Символы: КОИ-8.....	23
38. Символы: ISO8859-5 (ГОСТ-основная).....	24
39. Символы: WIN1251.....	25
40. Символы: UNICODE, UTF-8.....	25
41. Big-endian и Little-endian.....	27
42. Представление строк.....	27
43. История развития ЭВМ.....	28
44. История ЭВМ в СССР/РОССИИ.....	28
45. Канальная организация.....	29
46. Раздельные шины.....	29
47. Общие шины.....	30
48. Мультиплексирование шин.....	30
49. Мультипроцессорность: UMA - Uniform Memory Access.....	30
50. Мультипроцессорность: Коммутатор.....	31
51. Мультипроцессорность: NUMA - Non-Uniform Memory Access	31
52. Современные коммерческие процессоры.....	32
53. CISC, RISC, VLIW.....	32
54. Характеристики памяти.....	32
55. Статическая vs Динамическая память.....	33
56. Адресуемая память.....	34
57. Адресуемая память с фиксацией строк и столбцов.....	34
58. Синхронная память SDRAM.....	35
59. Конструктивные особенности современной памяти.....	35
60. Память, ориентированная на записи.....	36
61. Память, с последовательным доступом*.....	37
62. Структура ассоциативного запоминающего устройства.....	37
63. Кэш память.....	37
64. Пирамида памяти.....	38
65. Влияние промахов кэш-памяти.....	38
66. Сегментно-страничная виртуальная память.....	38
67. MMU и TLB.....	39

1. Операционные системы

Предназначена для замены работы оператора компьютерной системы.

Делится на *пользовательские, серверные, встроенные* ОС.

Примеры: Windows, Linux, Unix, Android, IOS, VXWoks, DOS, Гипервизоры.

Включают в себя ядро с подсистемами управления памятью и процессами и драйверы устройств.

Оператор принимал программу и исходные данные от программистов, запускал ее и считывал результат. Дисплей и клавиатура появились не сразу, поэтому приходилось пользоваться пультом управления.

Операционная система

Загружает в память команду => контролирует их исполнение => предоставляет результат в понятном для человека виде => можно несколько команд запускать одновременно

- Пользовательские – предоставляют комфорт работы для пользователя.
- Серверные – для обработки большого количества запросов от других систем.
- Встроенные – поддерживают функционал аппаратного обеспечения вычислительной системы
- Гипервизоры – операционные системы, которые управляют другими операционными системами (VMWare)
- ОС – содержит ядро, библиотеки, файлы.

2. История Unix/Linux

Название произошло от акронима (UNiplex Information and Computing System)

Unix разработали *Кен Томпсон, Денис Риччи, Руд Канадей*. Написали на ассемблере для вычислительной системы PDP-7. Позднее переписали на C на выч. сист. PDP-11.

Распространялась практически бесплатно, в следствии чего стала такой популярной.

Линукс разработал Линус Товальд. В качестве окружения пользовательских программ используется стандарт POSIX.

3. Современность

System V:

Базовой версией коммерческих операционных систем UNIX служит ядро ОС UNIX System V. Ядро имеет уникальную поддержку горячей замены аппаратных компонентов без перезагрузки ядра, высокую масштабируемость и развитую архитектуру.

Пример: Solaris, AIX, HP-UX

BSD:

Часто используют для организации сетевой инфраструктуры, так как поддерживает большое количество протоколов.

Пример: MacOS, FreeBSD, OpenBSD

Linux:

Богатый выбор операционных систем. Отдельно распространяются десктопные и серверные варианты, различающиеся набором пакетов по умолчанию.

Пример: RedHat, Ubuntu, Debian, Gentoo

4. Ядро *NIX

Служебные операции выполняются в ядре. Ядро – набор подсистем, которые управляют оборудованием и программами пользователя.

Основные подсистемы: управление памятью, управление процессами и потоками, виртуальная файловая система, сетевая подсистема, драйвера.

Адресное пространство разбито на сегменты.

- Code – там выполняется сама программа
- Data – хранит данные для работы программы
- Stack – для хранения адресов
- Heap
- Библиотечный сегмент
- ...

Программы взаимодействуют между собой при помощи ядра и интерфейса системных вызовов.

5. Файловая система

Root – корень иерархии верхний элемент файловой системы.

Логические разделы дисковых накопителей могут подключаться в файловую систему в точках монтирования – созданных администратором директорий, в которых при подключении их файловая иерархия заменяется на расположенную на разделе диска.

В каждой директории обязательно есть файл “.” – ссылка на себя и “..” – ссылка на директорию выше по файловой иерархии.

- */usr/bin* – утилиты и программы, поставляемые вместе с ОС.
- */var* – находятся системные журналы и рабочие файлы системы.
- */tmp* – хранит временные файлы.
- */proc* – процессы пользователя.

Абсолютный путь – путь от корня.

Относительный путь – путь от текущей директории, откуда была запущена программа пользователя.

Inode – уникальный номер в файловой системе для каждого файла.

Если два файла имеют один и тот-же inode в них одинаковое содержимое (жесткая ссылка).

Символическая ссылка – файл содержащий абсолютный или относительный путь до файла.

6. Права доступа к файлам

Права доступа разбиты на группы:

Тип файла	Права для владельца	Права для группы	Права для остальных
-	r w -	r - -	r - -

Типы файлов: файл, директория, символическая ссылка.

R – чтение

W – запись

X – исполнение/ можно войти для директории

7. Способы задания прав

Права принято обозначать в виде трех восьмеричных цифр.

Поэтому первый способ задания прав при помощи **chmod 644 имя файла**

Второй способ добавлять или убирать права при помощи буквенных последовательностей иго прибавлением к ним тех прав, которые мы хотим

Chmod ug = wx имя файла

8. Потoki stdin(0), stdout(1), stderr(2)

При запуске пользовательской программы системный загрузчик по умолчанию открывает для процесса потоки ввода-вывода и присваивает им номера.

Stdin(0) – поток стандартного ввода

Stdout(1) – поток стандартного вывода

Stderr(2) – поток вывода сообщений об ошибках.

По умолчанию потоки связаны с виртуальным терминалом, откуда была запущена программа.

9. Интерпретатор команд

Всеми командами управляет интерпретатор команд – shell.

Позволяет организовывать выполнение команд, задавать им переменные окружения, связывать потоки ввода вывода, использовать скрипты (новые написанные команды)

В настоящее время shell используют в серверных ОС.

В UNIX самая популярная оболочка bash.

Bash связывает потоки программы с терминалом и дальше если надо подключиться к потокам программы необходимые файлы или свяжет команды между собой.

10. Перенаправление потоков std...

—> *file* – перенаправить stdout в *file*

—>> *file* – добавить stdout к *file*

—2>> *file* – добавить stderr к *file*

—<< EOF – записать в stdin из терминала до символов EOF

—| - конвейер

— `ls | sort` – перенаправить `stdout` команды `ls` на `stdin` команды `sort`

11. Фильтры

Утилиты для фильтрации файлов – фильтры

Grep – осуществляет выборку строк совпадающих с шаблоном, заданным регулярными выражениями.

Sort – осуществляет сортировку строк по заданному условию (по умолчанию в алфавитном порядке)

12. Регулярные выражения

Регулярное выражение определяет шаблон, которому должна соответствовать строка в тексте.

`^` - начало строки

`$` - конец строки

`*` - любая последовательность символов

`[]` – или

`""` – экранировка

И тд...

13. Команды

Команда	Назначение и синтаксис
<code>mkdir</code>	<code>mkdir [-m mode] [-p] dir...</code>
<code>echo</code>	<code>echo [string]...</code>
<code>cat</code>	<code>cat [-n] [file...] [-]</code>
<code>touch</code>	<code>touch [-am]... file...</code>
<code>ls</code>	<code>ls [options] [file/dir]...</code>
<code>pwd</code>	<code>pwd</code>
<code>cd</code>	<code>cd [argument]</code>
<code>more</code>	<code>more [file...]</code>
<code>cp</code>	<code>cp [options] SOURCE ... DEST</code>
<code>rm</code>	<code>rm [options] [file/dir]</code>
<code>rmdir</code>	<code>rmdir [dir]</code>
<code>mv</code>	<code>mv [-fi] SOURCE ... DEST</code>
<code>head</code>	<code>head [-num] [file...]</code>
<code>tail</code>	<code>tail [-/+num] [-bcl] [file...]</code>
<code>sort</code>	<code>sort [-unr] [-k num] [file...]</code>
<code>grep</code>	<code>grep [-v] regexp [file...]</code>
<code>wc</code>	<code>wc [-c -m] [-lw] [file...]</code>

— `mkdir` – создание директории

— `echo` – вывод символов на стандартный вывод

- cat – слияние и вывод файлов на стандартный вывод
- touch – создание или изменение времени модификации файла
- ls – вывод списка файлов в директории
- pwd – вывод текущей директории в стандартный вывод
- cd – смена текущей директории на указанную
- more – постраничная распечатка файла или стандартного ввода
- cp – копирование файлов и директорий
- rm – удаление файлов
- rmdir – удаление директорий
- mv – перемещение файлов
- head – вывод указанного количества строк с начала файла или stdin
- tail – вывод указанного количества строк с конца файла или stdin
- sort – сортировка
- grep – фильтр
- wc – подсчет кол-ва строк/ слов/ символов в файле или stdin

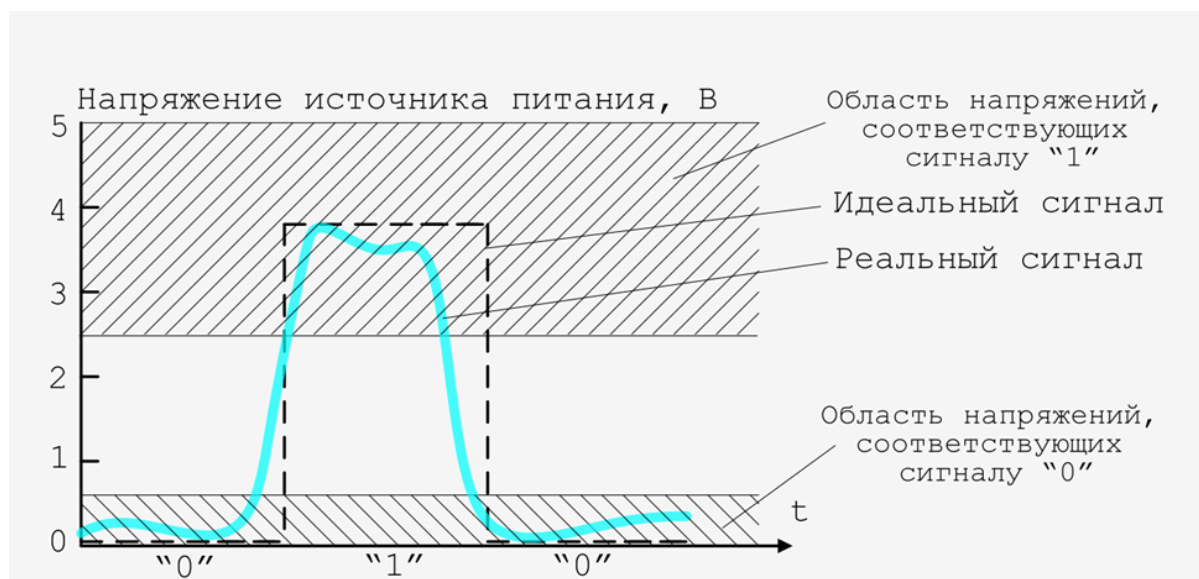
14. Аналоговые ЭВМ

Вместо величин таких как высота, давление и тд. Можно использовать их аналог (так как иначе потом тяжело с ними работать). В современной ВТ это напряжение.

Значению высота ставится значение напряжения с датчика высоты. Основной недостаток такой ЭВМ – большие габариты. При этом не всегда удаются преобразовать физическую величину в аналог и обратно. Допустим, чтобы отличить 200м от 201м требуются высокоточные приборы.

За счет того, что аналоговый сигнал непрерывен, его интегрирование и дифференцирование происходит быстрее чем у цифровых. Поэтому для отдельных задач аналоговые ЭВМ подходят лучше. (Системы реального времени, измерительные устройства)

15. Цифровые ЭВМ



Вопрос о недостаточной точности аналоговых ЭВМ привел к созданию цифровых ЭВМ. Используется сигнал, у которого 2 значения, либо 0 либо 1.

При передаче внутри ЭВМ сигнал может искажаться, если он будет сильно искажен, то будут возникать ошибки, но их можно детектировать либо вручную, либо при помощи специальных программ. Часто ошибки появляются при работе с памятью и передачей данных. Импульсы объединяют в группы, что позволяет закодировать информацию кодовой посылкой в виде одного или нескольких чисел.

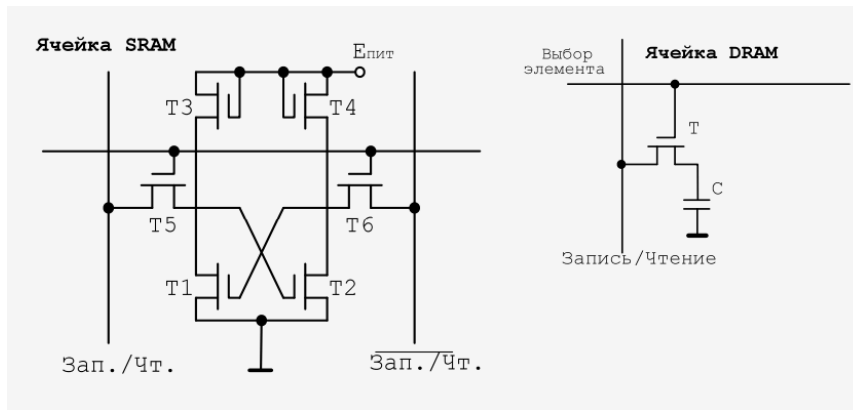
Цифровое представление упрощает необходимые элементы для вычислительной машины.

16. Функциональные элементы ЭВМ

- *Логические элементы* – выполняют логические функции (инвертор, И, ИЛИ, И-НЕ, ИЛИ-НЕ, XOR, и тд) несколько входов и строго один выход. Их можно объединять.

- *Элементы хранения* (DRAM/SRAM) – оперативная память, в которую сохраняется информация, пока ЭВМ включена.

В современных вычислительных системах используется два типа памяти: *динамическая* и *статическая*



DRAM – динамическая память, информация хранится на емкости, которая находится в цепи зарядов транзистора. Для одного бита один транзистор – это плохо, но плохо, что транзисторы потихоньку разряжаются и их надо подзаряжать, иначе произойдет потеря информации. Компьютер раз в определенное время сканирует всю оперативную память и считывает каждую ячейку, возобновляя заряд.

SRAM – статическая память, не требуется регенерация емкости, за счет особой конструкции из шести транзисторов, является более быстрой, но и больше. Используется для кэша например.

- *Элементы хранения (триггеры, регистры)* – триггер = защелка, с помощью которой можно хранить и управлять информацией.

D-триггер представляет собой ячейку информации, хранящую один бит информации.

Триггеры объединяются в регистры. Регистры образуют промежуточное хранение информации. Обозначается прямоугольником.

- *Провода, шины* – нужны для передачи информации. Для передачи 4-х бит информации нужно минимум 6 проводов (4 информационных, земля и управляющий)

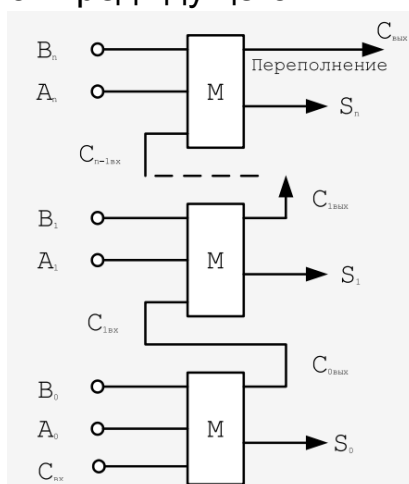
При передаче по проводам на дальние расстояния возникают ошибки, так как электромагнитные поля мешают, поэтому в таких случаях используют витую пару.

Обозначается широким проводником.

- *Вентиль* - Передачу данных нужно осуществлять строго заданное время. Для этого предусмотрено логическое устройство,

называемое вентилем. Либо открыт, либо закрыт (Работает по логике И). Есть управляющий сигнал и информационный. Обозначается ромбиком

- *Сумматор* – для выполнения простейших операций таких как сложение и вычитание нужен сумматор для сложения у логического элемента должно быть 3 входа. 2 отвечают за значения в ячейках и один за перенос. Перенос из старшего разряда то фиксируется переполнение. Не очень быстродейственно, так как сложения каждого разряда зависит от предыдущего.



- *Тактовые генераторы* – сердце ЭВМ. Задаёт ритм работы. Так как все операции могут происходить только в заданное время, определяемое размером такта. Характеризуются частотой, периодом импульса и фазовым сдвигом начала импульса. При использовании нескольких тактовых генераторов они должны быть синхронизированы, сдвиг фаз равен нулю.

Современные тактовые генераторы работают на частоте 3ГГц.

17. Первая ЭВМ: Калькулятор

Состоит из двух регистров X и Y, хранящих результат ввода, АЛУ, которая может выполнять простейшие арифметические действия и логические операции, шин и вентилях, устройств управления.

Дисплей всегда отображает X

После нажатия на цифру значение X записывается в Y, обнуляется.

После на АЛУ подается 0 и выбранная цифра и записывается в X (цифра + 0 = цифра)

После нажатие следующих цифр происходит то же самое, но при этом X надо сдвигать на разряд влево каждый раз.

После указанной команды X и Y передаются в АЛУ
Результат записывается в X

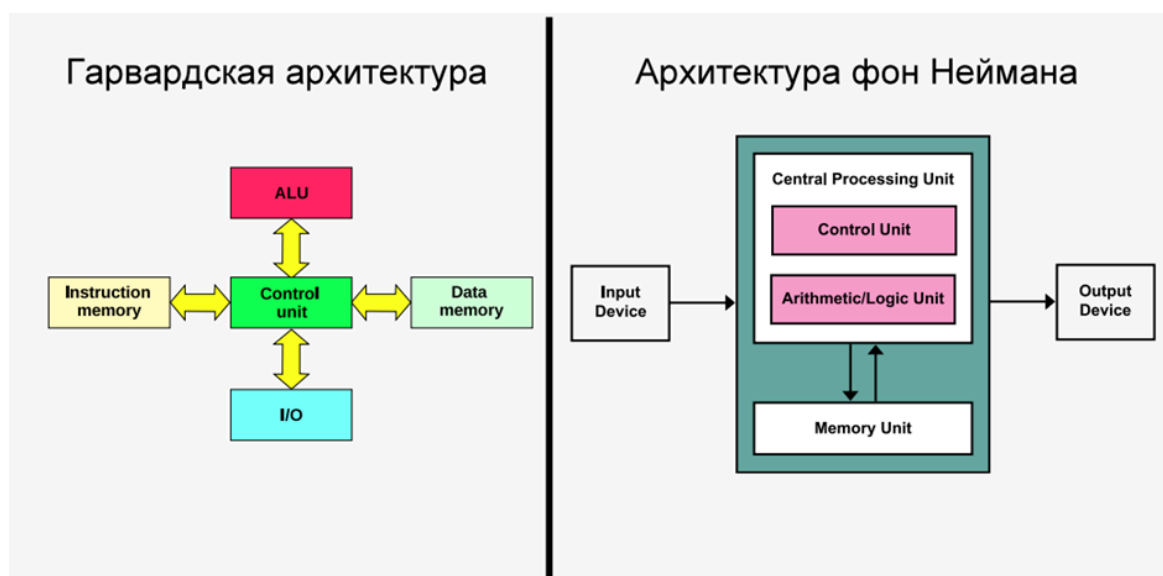
18. Архитектура ЭВМ Гарвардская, фон Неймана

Главное отличие между ними – работа с памятью.

В гарвардское центральное устройство – Control Unit – управляющее устройство. Все остальные устройства подключены к нему и взаимодействуют через него.

Плюс такой архитектуры в том, что нельзя нарушить логику программы, пытаясь исполнить данные или использовать инструкции в качестве данных.

В фон Неймана наоборот. Все в общей памяти. Нельзя без доп анализа определить, что находится в ячейке: инструкция или данные. Но такая конструкция проще, тк обращение к памяти едино. БЭВМ – на фон Неймана.



19. Структура БЭВМ-NG

БЭВМ включает в себя функциональные блоки и регистры:

- ❑ Память – 2048 (2^{11}) ячеек в каждой 16 разрядов. Для обращения к ней нужен 11 разрядный **AR** в который нужно поместить адрес, прежде чем обратиться к памяти.
- ❑ АЛУ. Может выполнять несколько операций: сложение, логическое умножение, инверсия и прибавление единицы.

- **DR** (*Data Register*) – служит для чтения или записи данных в/из ячеек памяти 16 разрядов.
- **IP** (*Instruction Pointer*) – счетчик команд 11 разрядов, хранит адрес следующей ячейки.
- **BF** (*Buffer Register*) – хранит промежуточные значения 16 разрядов.
- **CR** (*Command Register*) – используется для хранения кода команды и декодирования операций 16 разрядов.
- **AC** (*Accumulator*) – БЭВМ аккумуляторного типа, поэтому все вычисления с данными производятся через него.
- **SP** (*Stack Pointer*) – 11 разрядный указатель стека.
- **IR** (*Input Register*) – клавишный 16 разрядный регистр в составе пульта оператора.
- **PS** (*Program State*) – 16 разрядный регистр состояния хранит биты управляющие работой БЭВМ.

20. Устройство управления:

Устройство управления разработано в виде микропрограммного устройства управления (МПУ) – простейшего компьютера, программа которого состоит из *микроопераций* – потактового изменения значений вентилей БЭВМ, которые производят вычисления в АЛУ, пересылку данных между регистрами и простейшие проверки. Код программы для МПУ – *микрокод*.

Исполнение в МПУ машинной команды – *цикл команды*.

Цикл делится на 5 частей:

1. *Цикл выборки команды* – загружает исполняемую команду в CR и частично декодирует. Выполняется для каждой команды.
2. *Цикл выборки адреса* – для обработки адресных команд и выборки адреса операнда.
3. *Цикл выборки операнда* – для тех команд, в которых это необходимо, размещает второй операнд в DR (первый всегда аккумулятор)
4. *Цикл исполнения* – исполняет выбранную команду.
5. *Цикл прерывания* – выполняется в случае, если разрешены прерывания и устройство готово к обмену.

Для обеспечения работы оператора БЭВМ в ней предусмотрена микропрограммная реализация *циклов пультовых операций*:

- ▷ *Ввод адреса* – адрес из клавишного регистра вводится в счетчик команд.

- ▷ *Чтение* – читает информацию по заданному IP.
- ▷ *Запись* – информация из IP записывается в память по заданному адресу.
- ▷ *Пуск* – сброс состояния БЭВМ и выполнение программы.

21. Адресуемая память БЭВМ:

БЭВМ имеет адресную память.

Такая память выбирает одну из ячеек, которая соответствует коду адреса, и операция с памятью проводится с выбранной ячейкой.

По верхней шине адреса поступает адрес на дешифратор адреса далее активизируется одна из линий. После этого приходит сигнал о нужной операции и активизируется нужная часть схемы

2048 16-ти разрядных ячеек адресуемой памяти.

22. АЛУ, коммутатор, блок признаков результата:

АЛУ имеет два входа – левый и правый. На каждом входе есть инвертор. Есть отдельный вентиль для увеличения на 1 (INC).

Основная часть АЛУ состоит из сумматора или логического умножения. Выбор операции происходит с помощью вентиля SORA (Sum OR And).

В коммутатор поступают 18 разрядов с АЛУ (16 – результат, бит переноса, бит переноса из 14 разряда в 15). В нем осуществляется прямая передача данных, обмен байтов слова, арифметические и логические сдвиги.

Выход коммутатора идет на блок признаков результата.

N – бит отрицательного числа

Z – бит нуля

V – бит переполнения знаковых чисел

C – бит переноса беззнаковых чисел

23. Форматы команд:

- Безадресные команды:
Не содержит номер ячейки. Код операции всегда равен 0000.
0000 | 12 битов (расширение кода операции)
- Команда ввода-вывода максимум 256 устройств
0001 | приказ на 4 бита | 8 битов на на устройство

Приказ: ввод, вывод, проверка и сброс флага устройства

- Команда ветвления используется для организации переходов по заданному условию.

1111 | Расш. КОП на 4 бита | 8 бит на смещение

Безадресная команда

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	Расширение КОП											

Команда ввода-вывода

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	Приказ				Устройство							

Команда ветвления

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	Расш. КОП				Смещение							

- Адресные команды
 - С прямой абсолютной адресацией
КОП | 0 | Адрес
 - С относительной адресацией
КОП | 1 | Режим на 3 бита | смещение на 8 бит (относительно IP)
 - С непосредственной загрузкой операнда в аккумулятор.
КОП | 1111 | Число на 8 бит (операнд)

... с прямой абсолютной адресацией

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
КОП				0	Адрес										

... с относительной адресацией

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
КОП				1	Режим			Смещение							

... с непосредственной загрузкой операнда

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
КОП				1	1	1	1	Число							

24. Адресные команды

Тупо, наверное, надо привести примеры с мнемоникой и описать что делает команда.

Про работу SUB можно сказать, что такой команды напрямую нет, поэтому реализуется через $AC + (^M + 1)$

Наименование	Мнемон.	Код	Описание
Логическое умножение	AND M	2XXX	$M \& AC \rightarrow AC$
Логическое или	OR M	3XXX	$^M \& ^AC \rightarrow AC$
Сложение	ADD M	4XXX	$M + AC \rightarrow AC$
Сложение с переносом	ADC M	5XXX	$M + AC + C \rightarrow AC$
Вычитание	SUB M	6XXX	$AC - M \rightarrow AC$
Сравнение	CMP M	7XXX	Установить флаги по результату $AC - M$
Декремент и пропуск	LOOP M	8XXX	$M - 1 \rightarrow M$; Если $M \leq 0$, то $IP + 1 \rightarrow IP$
Резерв		9XXX	
Загрузка	LD M	AXXX	$M \rightarrow AC$
Обмен	SWAP M	BXXX	$M \leftrightarrow AC$
Переход	JUMP M	CXXX	$M \rightarrow IP$
Вызов подпрограммы	CALL M	DXXX	$SP - 1 \rightarrow SP$, $IP \rightarrow (SP)$, $M \rightarrow IP$
Сохранение	ST M	EXXX	$AC \rightarrow M$

25. Безадресные команды

Наименование	Мнемон.	Код	Описание
Нет операции	NOP	0000	Место для точек отладки, «патч» программы
Останов	HLT	0100	Отключение ТГ, переход в пультовый режим
Очистка аккумулятора	CLA	0200	$0 \rightarrow AC$
Инверсия аккумулятора	NOT	0280	$\wedge AC \rightarrow AC$
Очистка рег. переноса	CLC	0300	$0 \rightarrow C$
Инверсия рег. переноса	CMC	0380	$\wedge C \rightarrow C$
Циклический сдвиг влево	ROL	0400	AC и C сдвигается влево. $AC_{15} \rightarrow C, C \rightarrow AC_0$
Циклический сдвиг вправо	ROR	0480	AC и C сдвигается вправо. $AC_0 \rightarrow C, C \rightarrow AC_{15}$
Арифметический сдвиг влево	ASL	0500	AC сдвигается влево. $AC_{15} \rightarrow C, 0 \rightarrow AC_0$
Арифметический сдвиг вправо	ASR	0580	AC сдвигается вправо. $AC_0 \rightarrow C, AC_{15} \rightarrow AC_{14}$
Расширение знака байта	SXTB	0600	$AC_7 \rightarrow AC_{15} \dots AC_8$
Обмен ст. и мл. байтов	SWAB	0680	$AC_7 \dots AC_0 \leftrightarrow AC_{15} \dots AC_8$

Наименование	Мнемон.	Код	Описание
Инкремент	INC	0700	$AC + 1 \rightarrow AC$
Декремент	DEC	0740	$AC - 1 \rightarrow AC$
Изменение знака	NEG	0780	$\wedge AC + 1 \rightarrow AC$
Чтение из стека	POP	0800	$(SP)+ \rightarrow AC$
Чтение флагов из стека	POPF	0900	$(SP)+ \rightarrow PS$
Возврат из подпрограммы	RET	0A00	$(SP)+ \rightarrow IP$
Возврат из прерывания	IRET	0B00	$(SP)+ \rightarrow PS, (SP)+ \rightarrow IP$
Запись в стек	PUSH	0C00	$AC \rightarrow -(SP)$
Запись флагов в стек	PUSHF	0D00	$PS \rightarrow -(SP)$
Обмен вершины стека с аккумулятором	SWAP	0E00	$AC \leftrightarrow (SP)$

26. Представление чисел: фиксированная точка

1) Фиксированная *двоичная* точка может располагаться после 0 разряда двоичного числа (1011.).

Тогда будет стандартный перевод в 10 СС, умножением на $2^{\text{№ разряда}}$

2) *Двоичная* точка может располагаться правее (положительное смещение) (1011_.). В таком случае последний бит будет пустой, а их номера останутся, следовательно, при переводе в 10 СС по стандартному алгоритму, не получится представить нечетные числа. (По сути происходит сдвиг влево). Такая тема используется для кодирования больших чисел, но при этом шаг между числами будет больше, с каждым увеличением разряда

3) *Двоичная* точка может быть фиксирована с отрицательным сдвигом, тогда получатся вещественные числа, перевод будет осуществляться с отрицательными степенями.

4) Можно зафиксировать *десятичную* точку. Например, заранее сказать, что при переводе в 10 СС мы должны будем умножить полученное число на 0.1. Тогда мы сможем представлять числа с любой точностью.

Фиксация точки это лишь то, как мы хотим видеть числа в ЭВМ, порядок действий АЛУ и вычисления не меняются.

27. Представление беззнаковых целых чисел

Количество разрядов в разрядной сетке определяет ОДЗ.

Минимальное число – все разряды 0, максимальное – все 1.

Диапазон: $0 \leq X \leq 2^N - 1$.

28. Представление знаковых целых чисел

Существует несколько способов кодирования.

– *Прямое кодирование*: первый бит - знак, остальные обычные (0011 = 3, 1011 = - 3). Но в таком случае возникает двойной нуль. ОДЗ будет $[-(2^{n-1} - 1); 2^{n-1} - 1]$.

Прямой код в АЛУ не используется, т.к. необходима доп логика при определении знаков и больших по модулю чисел, что усложняет конструкцию АЛУ.

– *Обратный код*: инверсия числа. Проблема - двойной нуль.

– *Дополнительный код*: представляет отрицательные числа в виде дополнения до максимально возможного положительного числа + 1 в заданной разрядной сетке.

29. Представление знаковых чисел: дополнительный код

$M = b^n - K$. (M – дополнение к числу K , b – основание СС, n – кол-во разрядов). b^n – максимальное положительное число + 1.

$$M = ((b^n - 1) - K) + 1.$$

$(b^n - 1) - K$ - обратный код.

Для получения доп кода числа можно использовать поразрядное дополнение каждой цифры числа до максимально возможной. Например, поразрядное дополнение 12345 в 10СС будет 87654. Прибавим 1 и получим доп код. $12345 + 87654 + 1 = 100000 (b^n)$. За счет особенностей доп кода и отсутствия двойного нуля, отрицательных чисел на одно больше.

30. Перенос, Переполнение

В *беззнаковых* числах при прибавлении 1 к 15 или вычитании 1 из 0, происходит потеря значения числа, потому что возникает перенос (Carry). Загорается флаг C, который означает потерю значения.

В *знаковых* числах ошибка возникает между числами -8 и 7. При вычитании из -8 единицы или прибавлении 1 к 7 возникает переполнение (overflow). В разных архитектурах процессоров обозначается по-разному: O, V, OV, V.

АЛУ определяет переполнение по правилу: если поразрядные переносы в знаковом и старшем разряде одновременно отсутствуют или присутствуют (равны), значит переполнения нет, если присутствуют только в одном - значит есть переполнение.

1101 ← биты совпадают	1000 ← биты различны	0111 ← биты различны
+1101 -3	+1000 -8	+0111 7
0101 +5	1111 -1	0001 1
-----	-----	-----
10010 2	10111 7 (≠-9) ←ошибка	01000 -8 (≠8) ←ошибка
C=1 V=0	C=1 V=1	C=0 V=1

31. БЭВМ: представление чисел

Представление в разрядной сетке	Беззнаковые числа	Знаковые числа
0000 0000 0000 0000	0	0
0000 0000 0000 0001	1	1
...		
0111 1111 1111 1110	32766	32766
0111 1111 1111 1111	32767	32767
1000 0000 0000 0000	32768	-32768
1000 0000 0000 0001	32769	-32767
1111 1111 1111 1110	65534	-2
1111 1111 1111 1111	65535	-1

ОДЗ:

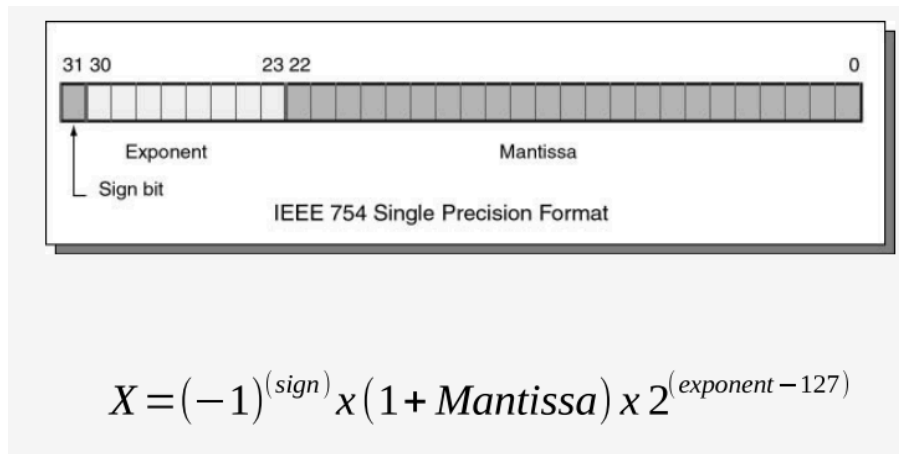
$0 \leq X \leq 2^{16} - 1$

$-2^{15} \leq X \leq 2^{15} - 1$

Если необходима более высокая разрядность числа, то при помощи команды учета переноса при сложении (ADC) возможны операции с 32-х разрядными и более числами.

32. Представление чисел с плавающей точкой

В плавающей точке положение десятичной точки не фиксируется за определенным разрядом, может меняться в зависимости от порядка числа.



$$(-1)^3 \times M \times 2^{(\text{э-смещение})}$$

Изображение: Skillbox Media

Здесь **З** — это знаковый бит, **М** — мантисса, **Э** — экспонента. А ещё у нас появилось смещение. Оно необходимо, чтобы экспонента могла принимать положительные и отрицательные значения, и в стандарте IEEE 754 представляет собой константу — число 127.

Если без смещения экспонента может принимать значения от 0 до 255, то со смещением от -127 до 128. Это нужно, чтобы представлять маленькие числа.

Представление числа состоит из мантиссы и порядка. *Мантисса* - дробная часть числа. Она внутри представлена *нормализованной* (начинается с 1).

Перед тем, как упаковать число, мантиссу нормализуют. Для хранения порядка числа, который указывает, на сколько разрядов была сдвинута запятая, используются отдельные разряды. Старший разряд - знак числа. Знак порядка образуется путем вычитания константы из значения порядка.

Форматы плавающей точки зависят от процессора. В современных используется устройство FPU (Floating point unit).

33. Представление логической информации

- 1-true, 0-false
- 16-ти разрядное число содержит 16 логических значений

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

ОДЗ: $X_i \in \{0,1\}$ где $0 \leq i \leq 15$

При команде AND производится 16 одновременных логических операций умножения. Полученный результат - ни знаковый, ни беззнаковый, это просто 16 отдельных логических значений.

34. Представление символьной и текстовой информации

Представление текстовой информации происходит с помощью кодовой таблицы. *Кодовая таблица* - соответствие каждому символу определенного порядкового номера или кода, чтобы можно было сохранять в ЭВМ или передавать по каналам связи. Для хранения графических начертаний символов существуют *шрифты*. Они хранят все изображения символов в заданном алфавите.

35. Символы: ASCII

ASCII Code Chart																									
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F									
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI									
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US									
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/									
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?									
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O									
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_									
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o									
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL									

ASCII расшифровывается как American Standard Code for Information Interchange. Содержит 128 символов, использует 8 бит. Из них 7 бит - для информации, 1 бит - бит контроля четности. В таблице присутствуют буквы, цифры, знаки препинания, спецсимволы и управляющие символы.

С помощью управляющих символов можно было управлять печатающей головкой. Так можно было, к примеру напечатать символ, вернуть каретку на символ назад и напечатать символ ударение, тем самым получить ударение над буквой.

36. Символы: ASCII (КОИ-7H0), КОИ-7H1 (РУС), КОИ7H2 (Mix)

20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F											
	!	"	#	¤	%	&	'	(	)	*	+	,	-	.	/											
30	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?										
40	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O										
50	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_										
60	Ю	А	Б	Ц	Д	Е	Ф	Г	Х	И	Й	К	Л	М	Н	О										
70	П	Я	Р	С	Т	У	Ж	В	Ь	Ы	З	Ш	Э	Щ	Ч											

Набор таблиц КОИ-7 (Код Обмена Информацией) был в СССР для русских символов.

H0: это US-ASCII, но \$ заменен на символ валюты ¤.

H1: все латинские буквы заменили на русские

H2: заглавные - латинские, строчные - русские.
 Кодировка 7-битная, как и ASCII, (тк старший бит - контроль четности).

Русские символы, идентичные по начертанию с латинскими, находились в тех же местах, что и английские.

37. Символы: КОИ-8

80	81	82	83	84	85	86	87	88	89	8A	8B	8C	8D	8E	8F
90	91	92	93	94	95	96	97	98	99	9A	9B	9C	9D	9E	9F
A0	A1	A2	A3	A4	A5	A6	A7	A8	A9	AA	AB	AC	AD	AE	AF
B0	B1	B2	B3	B4	B5	B6	B7	B8	B9	BA	BB	BC	BD	BE	BF
C0	C1	C2	C3	C4	C5	C6	C7	C8	C9	CA	CB	CC	CD	CE	CF
D0	D1	D2	D3	D4	D5	D6	D7	D8	D9	DA	DB	DC	DD	DE	DF
E0	E1	E2	E3	E4	E5	E6	E7	E8	E9	EA	EB	EC	ED	EE	EF
F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	FA	FB	FC	FD	FE	FF

При появлении Extended ASCII, где использовались все 8 разрядов байта, решили для русских заглавных и строчных символов использовать позиции верхней части таблицы. Кодировка называлась КОИ-8. Кроме русского алфавита были символы для рисования таблиц.

Если встречалось старое ПО, обрезающее старший бит для контроля четности, то русские символы перемещались в младшую часть таблицы и их можно было прочесть по сходным по начертанию английским символам.

КОИ-8 до сих пор используется как стандарт email.

38. Символы: ISO8859-5 (ГОСТ-основная)

	-0	-1	-2	-3	-4	-5	-6	-7	-8	-9	-A	-B	-C	-D	-E	-F
0-		0001	0002	0003	0004	0005	0006	0007	0008	0009	000A	000B	000C	000D	000E	000F
1-	0010	0011	0012	0013	0014	0015	0016	0017	0018	0019	001A	001B	001C	001D	001E	001F
2-	0020	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3-	0030	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>
4-	0040	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N
5-	0050	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^
6-	0060	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n
7-	0070	p	q	r	s	t	u	v	w	x	y	z	{		}	~
8-	0080															
9-	0090															
A-	00A0	Ё	Ђ	Ѓ	Є	Ѕ	І	Ї	Ј	Љ	Њ	Ћ	Ќ	-	Ў	Ц
B-	00B0	А	Б	В	Г	Д	Е	Ж	З	И	Й	К	Л	М	Н	О
C-	00C0	Р	С	Т	У	Ф	Х	Ц	Ч	Ш	Щ	Ъ	Ы	Ь	Э	Ю
D-	00D0	а	б	в	г	д	е	ж	з	и	й	к	л	м	н	о
E-	00E0	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю
F-	00F0	№	ё	ђ	ѓ	є	ѕ	і	ї	ј	љ	њ	ћ	ќ	џ	џ

ISO 8859-5 – 8-битная кодовая страница из семейства ISO-8859 для представления кириллицы.

Была создана в 1988 на базе “основной кодировки” (все русские буквы сохранили места, кроме заглавной Ё).

Российской адаптацией стандарта является ГОСТ Р 34.303-92, в котором кодировка названа КОИ-8 В1, однако в ней не установлены буквы нерусских алфавитов и коды управляющих символов.

Широко распространена в Сербии и Болгарии на UNIX-подобных системах. В России же широкого применения не нашла, в отличие от КОИ-8. На некоторых коммерческих UNIX-системах для русского до недавнего времени по умолчанию ставилась ISO 8859-5.

39. Символы: WIN1251

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
80	402 Ъ	403 Ѓ	201A ,	453 ř	201E „	2026 …	2020 †	2021 ‡	20AC €	2030 ‰	409 Љ	2039 ‹	40A Њ	40C Ќ	40B Џ	40F б
90	452 ђ	2018 ˆ	2019 ˇ	201C “	201D ”	2022 •	2013 —	2014 —	□	2122 ™	459 Ѝ	203A ›	45A Ў	45C Ў	45B Ў	45F Ў
A0	A0 ˆ	40E Ў	45E ˆ	408 Ј	A4 ˆ	490 ˆ	A6 ˆ	A7 ˆ	401 ˆ	A9 ˆ	404 ˆ	AB ˆ	AC ˆ	AD ˆ	AE ˆ	407 ˆ
B0	B0 ˆ	B1 ˆ	406 ˆ	456 ˆ	491 ˆ	B5 ˆ	B6 ˆ	B7 ˆ	451 ˆ	2116 №	454 ˆ	BB ˆ	458 ˆ	405 ˆ	455 ˆ	457 ˆ
C0	410 А	411 Б	412 В	413 Г	414 Д	415 Е	416 Ж	417 З	418 И	419 Й	41A К	41B Л	41C М	41D Н	41E О	41F П
D0	420 Р	421 С	422 Т	423 У	424 Ф	425 Х	426 Ц	427 Ч	428 Ш	429 Щ	42A Ъ	42B Ы	42C Ь	42D Э	42E Ю	42F Я
E0	430 а	431 б	432 в	433 г	434 д	435 е	436 ж	437 з	438 и	439 й	43A к	43B л	43C м	43D н	43E о	43F п
F0	440 р	441 с	442 т	443 у	444 ф	445 х	446 ц	447 ч	448 ш	449 щ	44A ъ	44B ы	44C ь	44D э	44E ю	44F я

Microsoft решили создать свою кодировку, которая создавалась на базе кодировок, использовавшихся в ранних русификаторах винды 1990-1991 совместно представителями “Параграфа”, “Диалога” и русского отделения микрософт.

40. Символы: UNICODE, UTF-8

Стандарт предложен в 1991 году организацией Консорциум Юникода. Кодировка позволяет кодировать большое количество символов из разных систем письменности.

Стандарт состоит из двух частей: универсального набора символов (Universal character set, UCS) и несколько форм представления для кодировок (Unicode transformation format, UTF).

Code point			UTF-8 encoded			
plane	row	column	byte 0	byte 1	byte 2	byte 3
0000,0000	0000,0000	0xxxxxxx	0xxxxxxx			
0000,0000	0000,0xxx	xyyyyyyy	110xxxxx	10yyyyyy		
0000,0000	xxxx,yyyy	yyzzzzzz	1110xxxx	10yyyyyy	10zzzzzz	
0000,wwxx	xxxx,yyyy	yyzzzzzz	1111,0www	10xxxxxx	10yyyyyy	10zzzzzz

Буква «А» → unicode \u0410 → 0000 0100 0001 0000

→ UTF-8 110 1 0000 1001 0000 → D0 90

Универсальный набор символов перечисляет допустимые по стандарту Юникод символы и присваивает каждому символу код в виде неотрицательного числа в 16-ричной форме с префиксом U+ (\u). Семейство кодировок определяет способы преобразования кодов символов для передачи в потоке или файле. В начальной части Юникод совпадает с ASCII, т.е младшие 7 битов

предваряются битом 0. Если символ превышает 7 разрядов, то он представляется в виде двух байтов, при этом первые 5 предваряются преамбулой 110, а следующий байт преамбулой 10, и оставшиеся 6 разрядов копируются во второй байт.

Кодировка UTF-8 преобразует code point Unicode в последовательность байтов. Принцип заполнения зависит от диапазона значения code point:

- *1 байт*: если значение code point лежит в диапазоне от 0 до 127 (совпадает с ASCII).
Формат: 0xxxxxxx (7 бит данных).
- *2 байта*: если значение code point лежит в диапазоне от 128 до 2047.
Формат: 110xxxxx 10xxxxxx (5 бит в первом байте и 6 бит во втором).
- *3 байта*: если значение code point от 2048 до 65535.
Формат: 1110xxxx 10xxxxxx 10xxxxxx (4 бит в первом байте и по 6 бит в остальных двух).
- *4 байта*: для символов с кодом выше 65536.
Формат: 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx (3 бит в первом байте и по 6 бит в остальных трёх).

Каждому русскому символу соответствует два байта.

Проблема UTF-8 в том, что один символ кодируется 1-4 байтами, что затрудняет нахождение длины строки.

Недостаток можно исправить, используя UTF-16 или 32, но потребуется больше памяти.

UTF-16:

- В UTF-16 большинство часто используемых символов (например, кириллица и латиница) кодируются фиксированными 2 байтами.
- Символы за пределами базовой многоязычной плоскости (например, эмодзи) занимают 4 байта (в виде пары суррогатов).
Таким образом, проблема длины строки в UTF-16 частично

решается, но всё равно встречаются символы переменной длины (2 или 4 байта).

UTF-32:

- В UTF-32 каждый символ кодируется **строго 4 байтами**, независимо от его диапазона. Это делает длину строки простой для вычисления — нужно просто разделить общий размер массива байтов на 4. Однако использование фиксированных 4 байтов на символ делает UTF-32 гораздо более "жадной" к памяти.

41. Big-endian и Little-endian

Аналогия с "Путешествиями Гулливера", где две империи воевали из-за спора с какого конца жрать яйцо — тупого или острого.

Возник спор: как хранить в памяти слова, в каком порядке располагать байты? Младший байт ставить первым или старший. Разницы с точки зрения архитектуры нет.

В современных ЭВМ используется Little-endian — младшие разряды слова располагаются в начальных байтах памяти. До сих пор встречаются несовместимости между различными архитектурами (например Intel и SPARC).

42. Представление строк

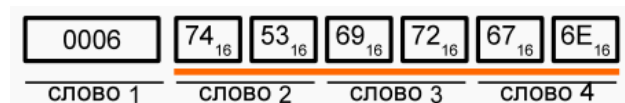
В БЭВМ не существует поддержки строк на уровне ЭВМ или библиотек. Нужно было выделять разное количество байт для строк, т.к. она ее длина — переменная величина. Есть два способа решения проблемы:

1) NUL terminated String

Признак конца строки — специальный символ, обычно NUL. Применяется в C и ему подобных ЯП.

2) Упаковка с длиной

Первое слово строки — длина строки. UTF-8 удобна тем, что позволяет определить количество байт, хранимых в длине строки и определить количество символов в ней.



43. История развития эвм

Все поколения разбиты по технологической базе.

Нулевое поколение - механические компьютеры (до 1945 года)

Налоговый сумматор и калькулятор на 4 действия.

Скорость 1 операция в секунду.

Первое поколение - электронные лампы (1945-1955)

Занимали много места и потребляли много энергии.

Невысокая скорость.

Машина Тьюринга, машина фон Неймана.

Второе поколение - транзисторы (1955-1965)

Транзистор позволил во много раз уменьшить размер одного элемента, соответственно уменьшился размер самих машин.

Сократилось потребление энергии.

Появились компании, которые занимались именно разработкой ЭВМ (IBM, DEC).

Третье поколение - интегральные схемы (1965-1980)

Схема содержала несколько транзисторов на одном кристалле.

Семейство System/360 (IBM), PDP-11 (DEC).

Появился рынок компьютеров.

Четвертое поколение - сверхбольшие интегральные схемы (1980-...)

IBM PC, Apple, Intel, IBM, DEC, HP - появилось много новых компаний. Компьютеры стали доступнее для народа.

За 40 лет почти ничего не поменялось, только компьютеры стали быстрее. Архитектура сильно не изменилась.

Пятое поколение - небольшие и «невидимые» компьютеры (1989-...)

Стиральная машина, холодильник, камера - однокристалльные компьютеры.

Интернет вещей, умный дом - также примеры.

44. История ЭВМ в СССР/РОССИИ

Первое поколение - электронные лампы

БЭСМ - 10000 ОП/С, 53 кВт потребление

Второе поколение - транзисторы

СССР не отставала в развитии ЭВМ.

Машины развивали скорость машин третьего поколения, 500000 оп/с

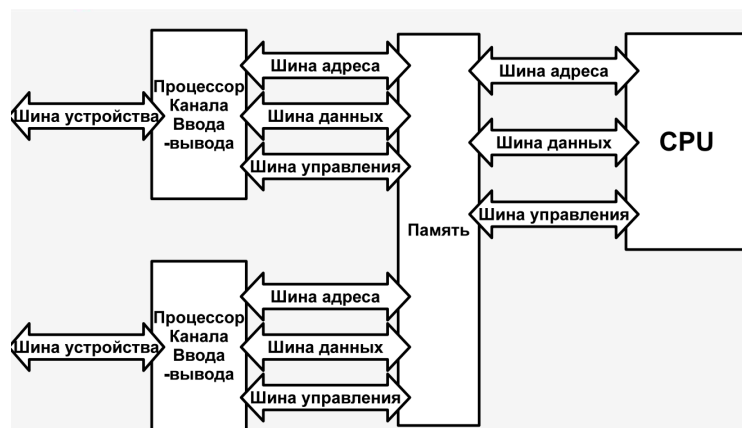
Третье поколение - интегральные схемы

Клонирование архитектуры американских компьютеров S/360 IBM и PDP-11

Четвертое поколение - сверхбольшие интегральные схемы

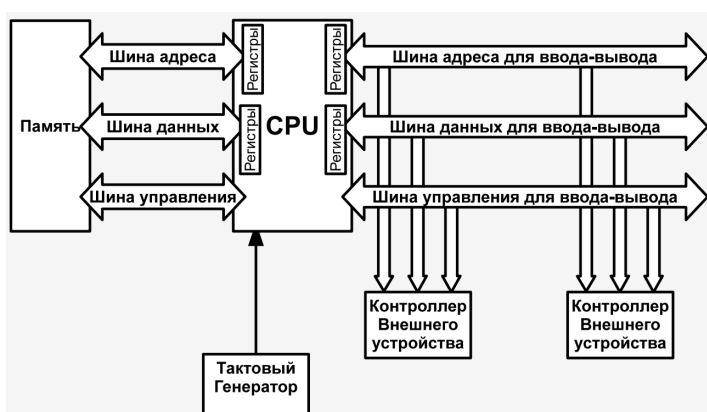
Эльбрус

45. Канальная организация



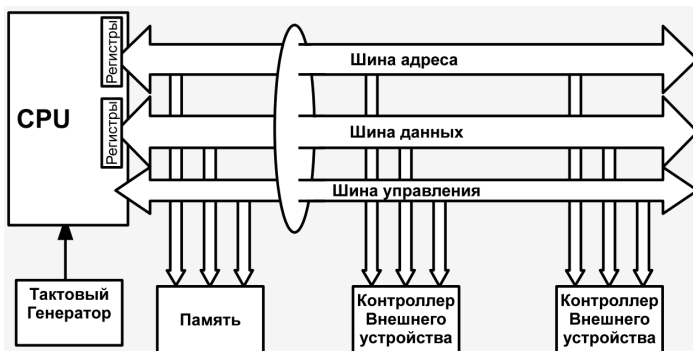
Канальная организация строилась вокруг памяти – центрального звено. Скорость всех блоков машины была примерно одинаковая. С внешними устройствами работали отдельные процессоры. 3 различных шины: адреса, данных, управления.

46. Раздельные шины



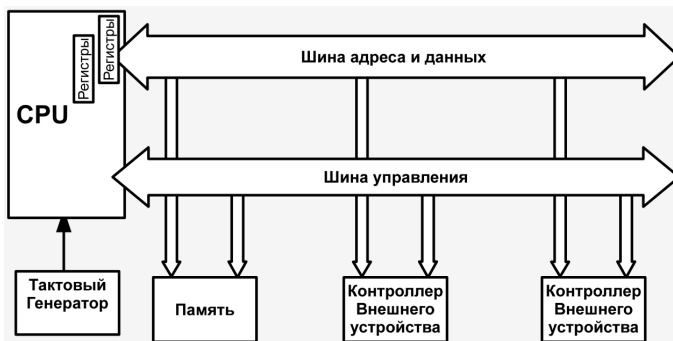
Через какое то время производительность процессора стала больше памяти. Центральным звеном архитектуры стал процессор. При этом контроллеры внешних устройств остались медленными. Появились раздельные шины для ввода-вывода.

47. Общие шины



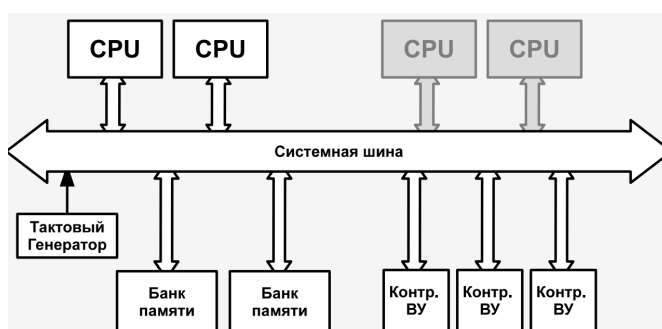
Из логики, что память и устройство ввода-вывода работают на примерно одинаковой частоте, можно повесить их на одну шину и не сильно потерять в производительности.

48. Мультиплексирование шин



Во время третьего поколения ЭВМ было решено передавать адреса и данные по одной шине для сокращения кол-ва проводов.

49. Мультипроцессорность: UMA - Uniform Memory Access



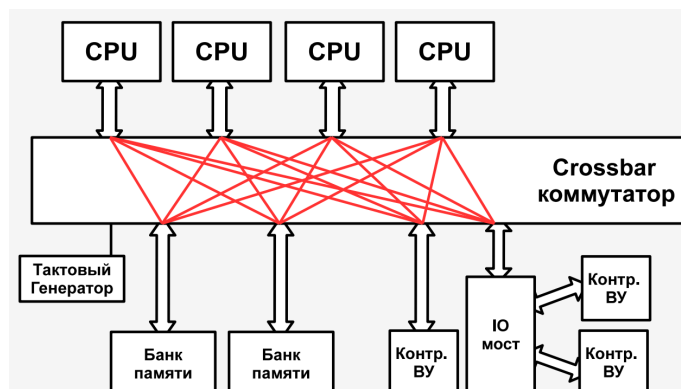
Системная шина - общее место для обмена информацией между блоками ЭВМ.

Причины использования нескольких процессов в одной машине - физические ограничения в повышении тактовой частоты.

Появилось много проблем из-за синхронизации работы нескольких процессоров.

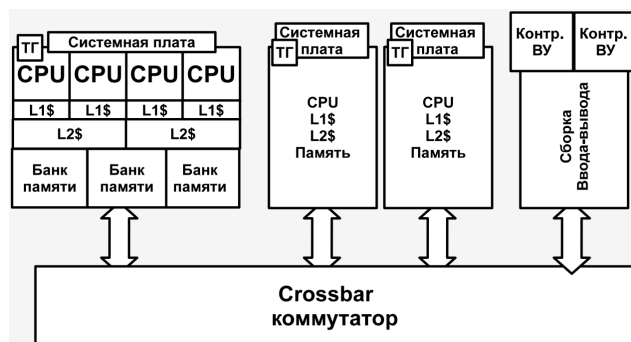
Но после добавления третьего процессора машина уже стала медленнее, из-за производительности шины.

50. Мультипроцессорность: Коммутатор



Все процессоры соединены отдельными каналами с каждым блоком ЭВМ (по сути - полный граф). Т.е. каждый процессор может обратиться к любому блоку параллельно, пока другой процессор обращается к другому блоку, тк это происходит по разным каналам. Еще появились мосты, которые подключаются к коммутатору и распределяют информацию уже на более медленные устройства. Те быстрые устройства подключаются напрямую к коммутатору, а медленные - через мост.

51. Мультипроцессорность: NUMA - Non-Uniform Memory Access



Такая архитектура позволяет работать с большим количеством процессоров.

Каждый процессор организован с локальной памятью в виде одной системной платы.

Все платы соединены коммутатором.

Процессор может работать внутри своей системной платы. До разной памяти (внутри своей платы, на другой платы) разное время доступа - поэтому и Non-Uniform.

Особенности:

1. Тактовый генератор содержится на каждой плате, поэтому процессоры могут работать на разной частоте.
2. Архитектура позволяет менять на горячую элементы ЭВМ.

52. Современные коммерческие процессоры

Разрядность от 16 до 64 бита

Тактовые частоты 500мгц-5ггц

Многопроцессорные 1-100 сри

Многоядерные 1-16 ядер

От 1гб до 1тб ОЗУ

Используют кэш память разных уровней

Суперскалярные

Используют архитектуру CISC, RISC, VLIW

53. CISC, RISC, VLIW

Complex Instruction Set Computer

Традиционные процессоры (Intel, amd), отягощенные совместимостью (поддерживают старые команды).

Reduced Instruction Set Computer

Простой набор инструкций, выполнение инструкции за такт. Убрали все ненужные команды. Но при этом производительность не сильно выше, чем у CISC, поэтому до сих пор все используют CISC.

Very Long Instructions Word

Несколько инструкций, упакованных в одну команду.

Упаковка операций в инструкцию ложится на компилятор.

54. Характеристики памяти

- Месторасположение (от него зависит цена памяти). Если вычисления происходят внутри процессора, то это быстрее всего.
 - процессорные
 - внутренние
 - внешние (ЖД, дисководы и тд)
- Емкость памяти
 - В метрических (Кило-) и двоичных (Киби-)
- Единицы пересылки
 - Слово, строка кэша, блок на диске. От единицы измерения зависит скорость передачи. Выгоднее прочитать несколько последовательных блоков.

- Метод доступа
 - Произвольный (адресный)
 - Ориентированный на записи (прямой)
 - Последовательный
 - Ассоциативный
- Быстродействие и временные соотношения
 - Время доступа T_d (то время, в течении которого память начнет отдавать первые результаты)
 - Длительность цикла (полное время от начала до конца)
 - Время чтения и время записи
 - Время восстановления T_v (приход в исходное состояние, сброс команды)
 - Скорость передачи информации
- Физический тип и особенности
- Стоимость

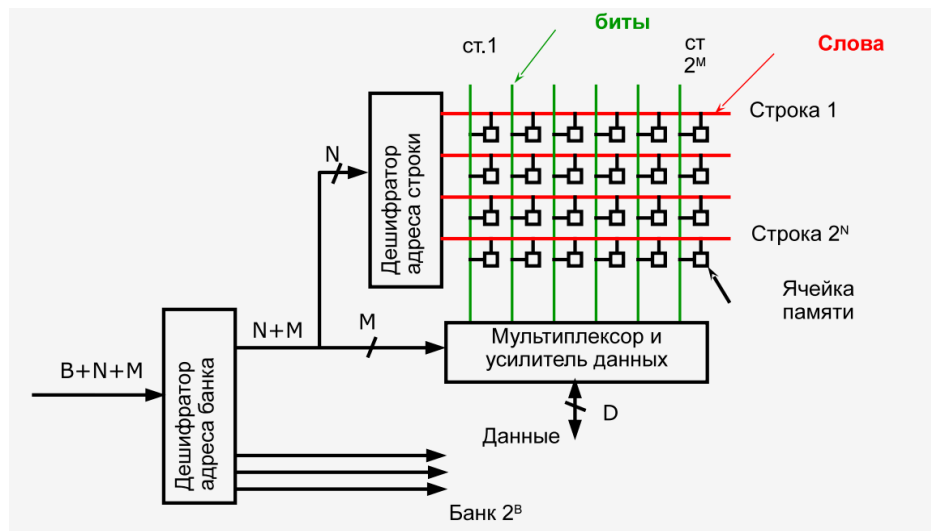
55. Статическая vs Динамическая память

Статическая дороже, занимает больше места.

DRAM – динамическая память, информация хранится на емкости, которая находится в цепи зарядов транзистора. Для одного бита один транзистор – это плюс, но плохо, что транзисторы потихоньку разряжаются и их надо подзаряжать, иначе произойдет потеря информации. Компьютер раз в определенное время сканирует всю оперативную память и считывает каждую ячейку, возобновляя заряд.

SRAM – статическая память, не требуется регенерация емкости, за счет особой конструкции из шести транзисторов, является более быстрой, но и больше. Используется для кэша например.

56. Адресуемая память

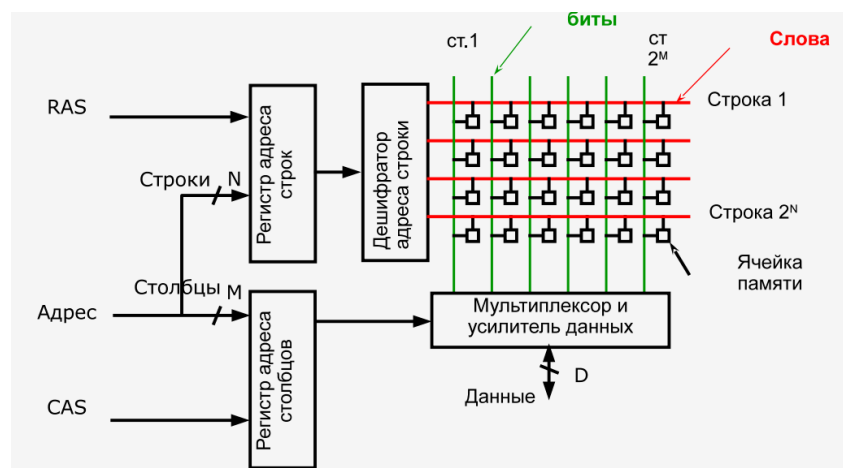


Адрес выражен числом. Есть банки памяти, дешифратор адреса строки и усилитель данных (выбирает нужный столбец) + дешифратор банка (адрес делится на порции и логически объединяет их, что дает полный адрес).

Мультиплексор осуществляет передачу данных неизменными с входа на выход.

Сначала проходит через дешифратор памяти, потом идет в дешифратор адреса и после в усилитель данных.

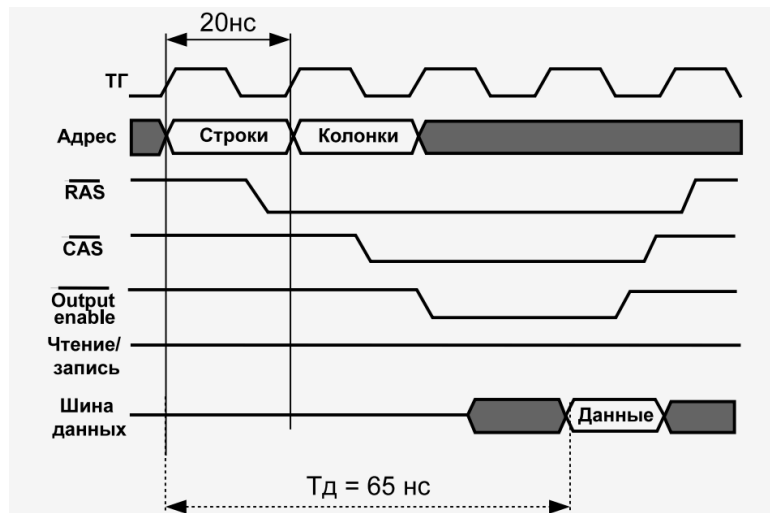
57. Адресуемая память с фиксацией строк и столбцов



Дешифраторы делать сложно поэтому адрес передается в виде адреса колонки и строки. Для обращения к памяти есть два регистра (адреса строк и столбцов). Подаем часть адреса, для выбора строки, потом другую половинку для столбца. Row Address Select / Column Address Select.

Столбцы выбираются дешифратором столбца
Строки дешифратором строк
Сделано для сокращения шины

58. Синхронная память SDRAM



Есть тактовый генератор (он достаточно на высокой частоте работает в современности, на слайде старый указан).

Есть шина адреса, два сигнала RAS и CAS, за выбор строки служит 0 (используется инвертированное значение сигнала).

Для чтения подаем адрес строки и выбор строки, потом сигнал адрес выбор колонки, потом уже сигнал вывода данных (время доступа).

За один такт одна команда.

Запись в такую память зависит от тактового генератора.

Читает сразу несколько ячеек подряд, на шину выводит заданное количество ячеек, чтобы можно было сэкономить на служебной информации.

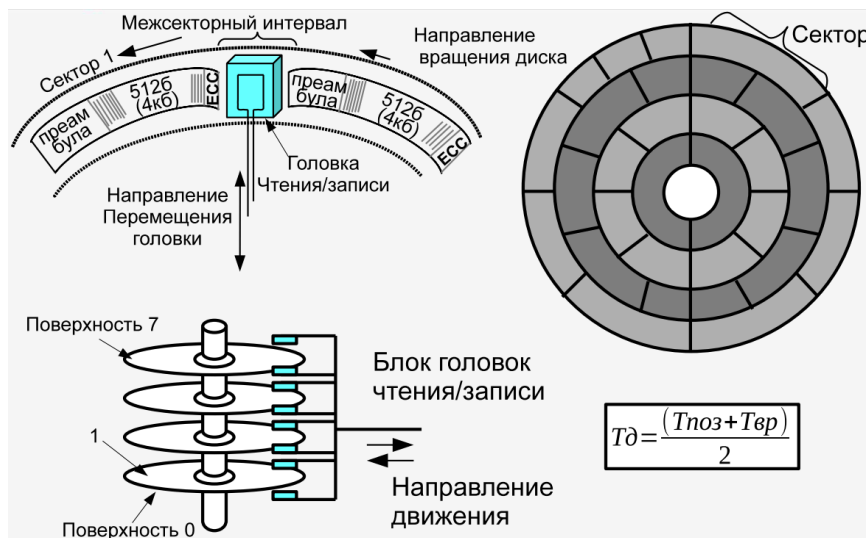
59. Конструктивные особенности современной памяти

- Burst mode - пакетный режим, когда много ячеек сразу выдается (этим режимом заполняется строка кеша).
- Double Data Rate (DDR) - передача данных по фронту и по спаду. На каждое изменение импульса частоты выдается значение. Частота становится в два раза выше.
- SPD - чип с конфиг информацией. Когда БИОС инициализирует память, он читает эту хуйню, что за железо,

его ID. Выгодно производить одинаковые чипы, в которых написано меньше инфы о памяти, что память меньшего размера, хотя фактически было не так.

- Interleaving - расслоение памяти, повышает производительность. Ставить память не отдельными чипами, а планками. Одновременно читается инфа из чипа сразу со всех чипов, увеличивает скорость.
- DDR4-2133 8192MB PC4-17000 индекс производительности. Быстрее, чем первый чип в 17000 раз.

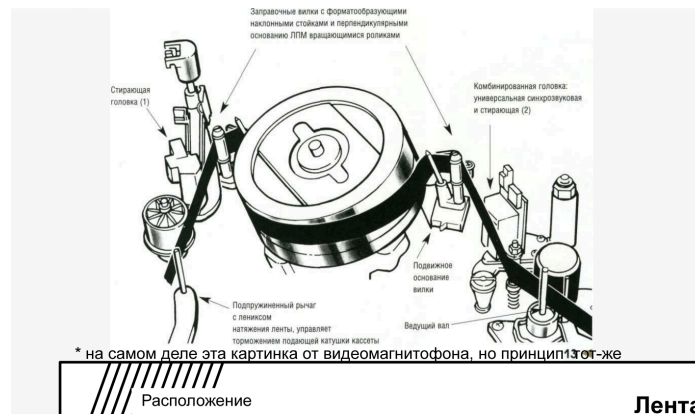
60. Память, ориентированная на записи



Пример – жесткий диск. Хранение в виде сектора. Блоки предваряются преамбулой, для определения где начало.

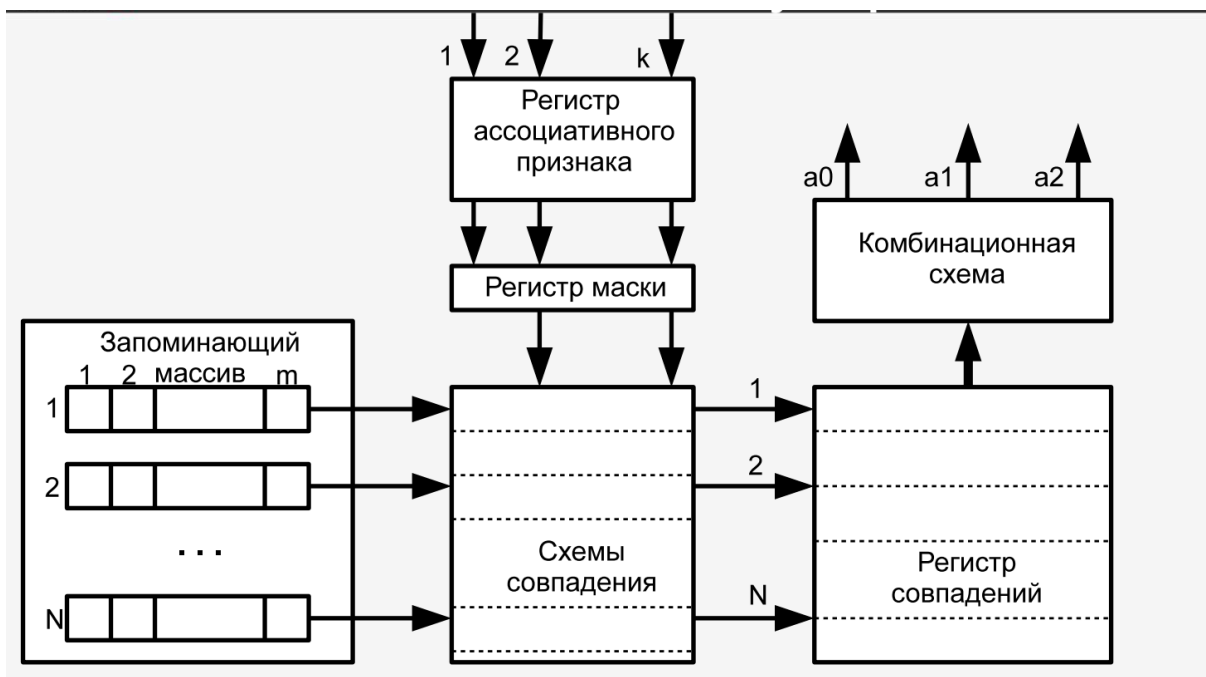
На каждой дорожке определенная последовательность блоков данных. Плотность информации разная и скорость чтения тоже. На внешней стороне диска одна скорость головки, на внутренней другая. Скорость чтения больше на внешней стороне.

61. Память, с последовательным доступом*



Ленточное устройство. Последовательно на ленте записана информация. Люди для увеличения скорости головки крутили, относительно ленты.

62. Структура ассоциативного запоминающего устройства



Ищем по ассоциативному признаку. Память разделена на память признаков и память данных.

Ячейка памяти содержит схему сравнения.

У кого признаки совпали, те срабатывают и записывают данные из массива.

63. Кэш память

Есть большая память и программа работает с ее кусками. В качестве признаков используется тег памяти. Одновременно с чтением из памяти выбирается строка из кэша и по шинам данных одновременно записывается в кэш 1 и 2 уровня. Потом вызывается не из памяти, а из кеша. При обращении к массиву из 16 элементов, к физической памяти вызов проходит всего 2 раза.

64. Пирамида памяти



The diagram shows a pyramid divided into seven horizontal layers, each representing a different level of memory hierarchy. From top to bottom, the layers are: CPU, L1 Cache, L2-L3 Cache, Основная память (Main Memory), SSD, Жесткие диски (Hard Disks), and Магнитные ленты (Magnetic Tapes). To the right of the pyramid is a table with six columns: Объем (Volume), Тд (Access Time), * (Hit Rate), Тип (Type), and Управл. (Management).

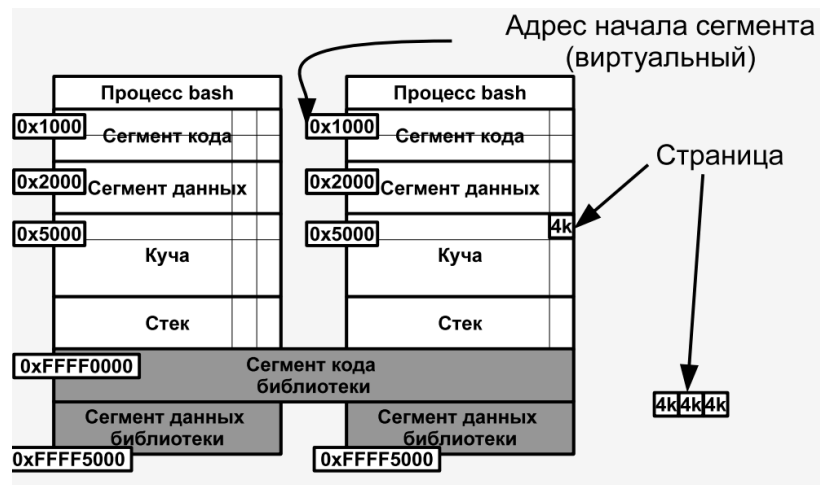
	Объем	Тд	*	Тип	Управл.
CPU	100-1000 б.	<1нс	1с	Регистр	компилятор
L1 Cache	32-128Кб	1-4нс	2с	Ассоц.	аппаратура
L2-L3 Cache	0.5-32Мб	8-20нс	19с	Ассоц.	аппаратура
Основная память	0.5Гб-4Тб	60-200нс	50-300с	Адресная	программно
SSD	128Гб-1Тб/drive	25-250мкс	5д	Блочн.	программно
Жесткие диски	0.5Тб-4Тб/drive	5-20мс	4м	Блочн.	программно
Магнитные ленты	1-6Тб/к	1-240с	200л	Последов.	программно

65. Влияние промахов кэш-памяти

Промахи кэша сильно снижают производительность.

Происходят, когда в кэше нет запрашиваемых данных. Поэтому приходится обращаться к обычной памяти и брать данные оттуда, что гораздо медленнее, см Пирамиду памяти. Поэтому производительность падает так сильно.

66. Сегментно-страничная виртуальная память



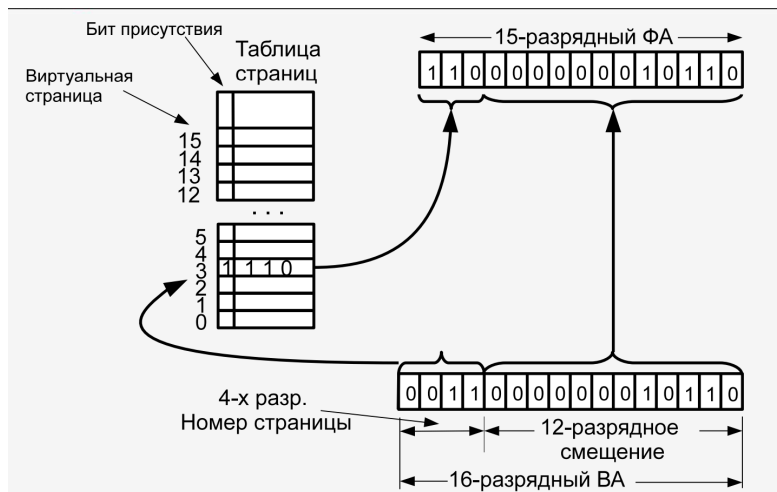
Память – набор сегментов, разбитых на страницы.
у сегмента свой адрес.

Виртуальная память. Каждый процесс работает в своем адресном пространстве, они могут пересекаться. Виртуальные адреса транслируются в физические. Одни и те же могут соответствовать разным физическим.

```
root@ra:~# pmap -x $$
5081:  /bin/bash
Address Kbytes  RSS Dirty Mode Mapping
08048000 1064   852    0 r-x-- bash
08152000    4     4    4 r---- bash
08153000   20    20   20 rw--- bash
08158000   20    20   20 rw--- [ anon ]
09c79000 1204  1204  1204 rw--- [ anon ]
b716a000   44    44    0 r-x-- libnss_files-2.23.so
b7176000    4     4    4 rw--- libnss_files-2.23.so
b74e5000  1724  1248    0 r-x-- libc-2.23.so
b7697000    4     4    4 rw--- libc-2.23.so

root@ra:~# pmap -x 6964
6964:  cat
Address Kbytes  RSS Dirty Mode Mapping
08048000   48    48    0 r-x-- cat
08054000    4     4    4 r---- cat
08055000    4     4    4 rw--- cat
09609000  132     8    8 rw--- [ anon ]
b724a000  136     4    4 rw--- [ anon ]
b739e000  2048   372    0 r---- locale-archive
b759e000  1724   964    0 r-x-- libc-2.23.so
b7750000    4     4    4 rw--- libc-2.23.so
```


67. MMU и TLB

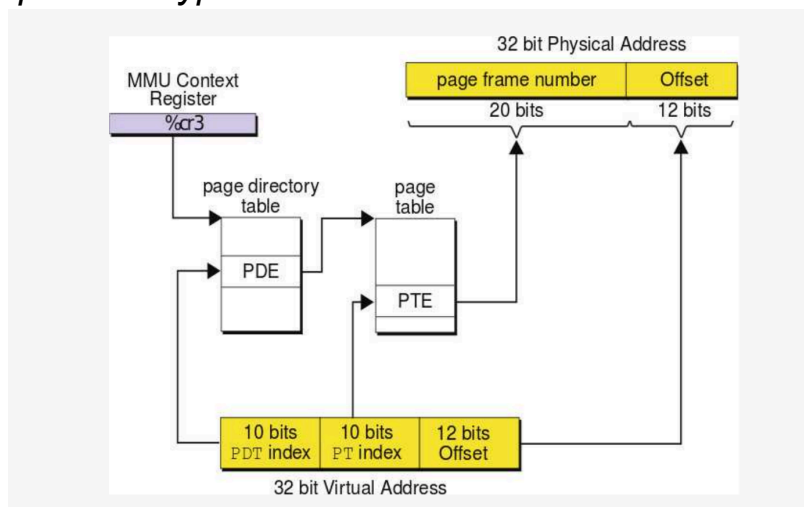


MMU (Memory Management Unit): трансляция адреса (устройство управления памятью).

Обычно берется виртуальный адрес, у него есть смещение, которое копируется в физический адрес.

Берется виртуальный адрес + номер процесса, поэтому один адрес может соответствовать разным физическим ячейкам.

Архитектура x86:



Есть 32бит виртуальный адрес и 32бит физ адрес, они преобразуются с помощью таблицы. Page dir table и page table – хранятся в оперативной памяти.

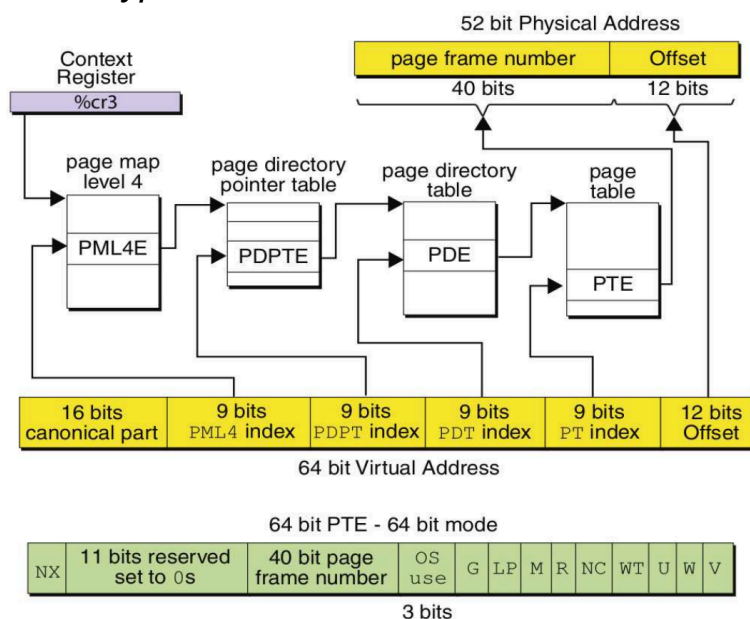
Чтобы обратиться к памяти, надо к физической памяти обратиться 3 раза:

Обращение к каталогу страниц: используется индекс PDT для выбора записи (Page Directory Entry, PDE), указывающей на таблицу страниц.

Обращение к таблице страниц: используется индекс PT для выбора записи (Page Table Entry, PTE), указывающей на физическую страницу.

Обращение к физической странице: с помощью смещения Offset.

Архитектура x64:



При обращении к памяти 1 раз – к физической памяти обращение происходит 5 раз.

Каталоги страниц разбивают таблицу на несколько уровней:

1. Первый уровень (PML4 в x64 или Page Directory в x86):
 - Делит адресное пространство на крупные блоки (например, 4 ГБ в x86 или 512 ГБ в x64).
 - Содержит указатели на второй уровень.
2. Второй уровень (PDPT в x64 или Page Table в x86):
 - Делит крупные блоки на более мелкие (например, 2 МБ или 4 КБ страницы).
 - Указывает на следующий уровень.
3. Третий и четвертый уровни (x64):
 - Делят память на еще более мелкие части.
 - Указывают на конкретные физические страницы.

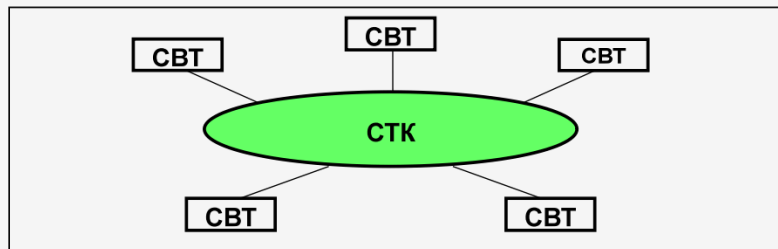
TLB:

- Кэширует часто используемые преобразования
- Раздельный для адреса и данных
- Организован в виде ассоциативной памяти

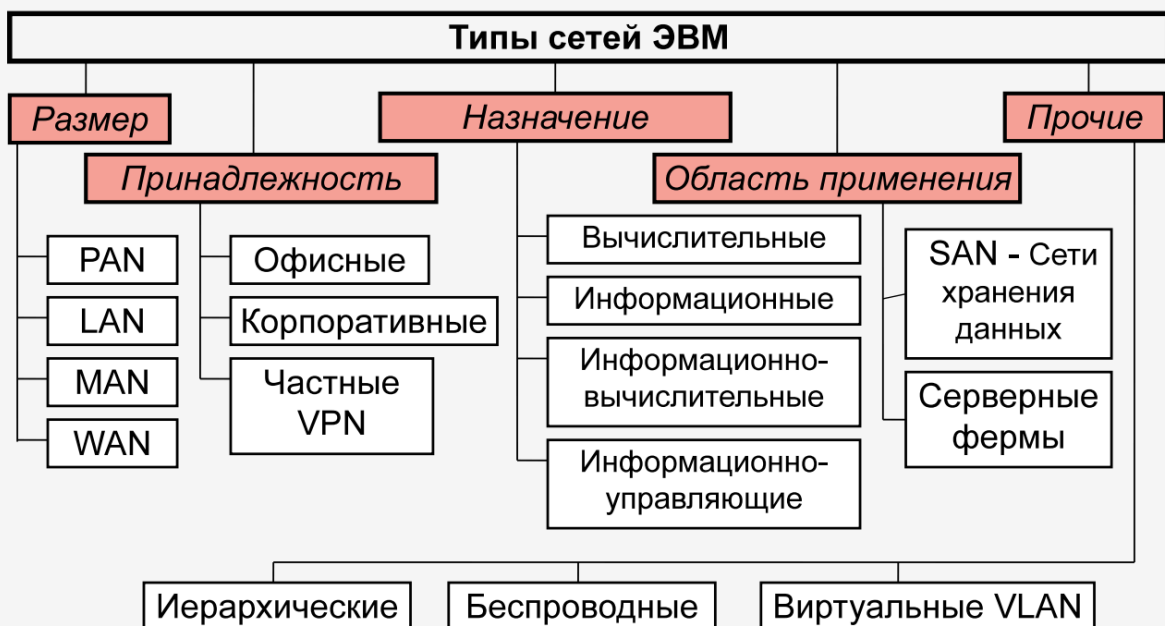
68. История сети Internet

- 1792 — Клод Шапп — оптический телеграф. Семафоры - 2 слова в минуту.
 - 1872 — Жан Бодо — телеграфный аппарат, код Бодо. Бод (бит в секунду).
 - 1895, 1897 — Попов — беспроводная передача между берегом и военным судном.
 - 1930 — Телетайп, сеть Телекс (TELEgraph + EXchange)
-
- 1957 — запуск первого спутника земли
 - 1958 — Advanced Research Projects Agency (ARPA).
 - 1963 — J.C.R. Licklider — первая концепция компьютерной сети
 - 1969 — ARPANET в ведущих лабораториях и исследовательских центра США
 - 1976 — Xerox — локальная сеть Ethernet
 - 1982 — ARPA — единый стек протоколов TCP/IP
 - 1983-84 — FidoNet и BBS, ARPANET → Internet
 - 1991 - Tim Berners-Lee, CERN, концепция WWW, первый http сервер

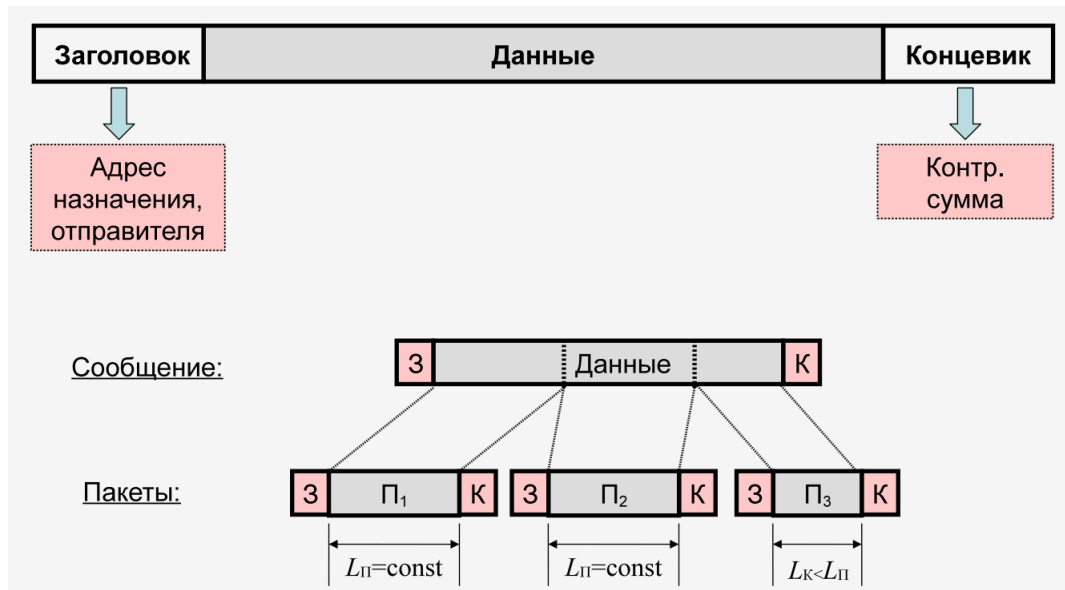
69. Понятие сети ЭВМ



- Средства вычислительной техники (СВТ): ЭВМ, вычислительные комплексы (ВК) и вычислительные системы (ВС) - реализуют обработку данных.
- Средства телекоммуникаций (связи) (СТК): совокупность каналов связи и каналообразующей аппаратуры - реализуют передачу данных.
- Сеть ЭВМ (вычислительная сеть, компьютерная сеть) = СВТ+СТК



70. Сообщение, пакет



Сообщения обычно длинные, поэтому их бьют на пакеты.

Размер пакета обычно 1500 байт.

Делить на пакеты надо, чтобы можно было вставлять свои слова.

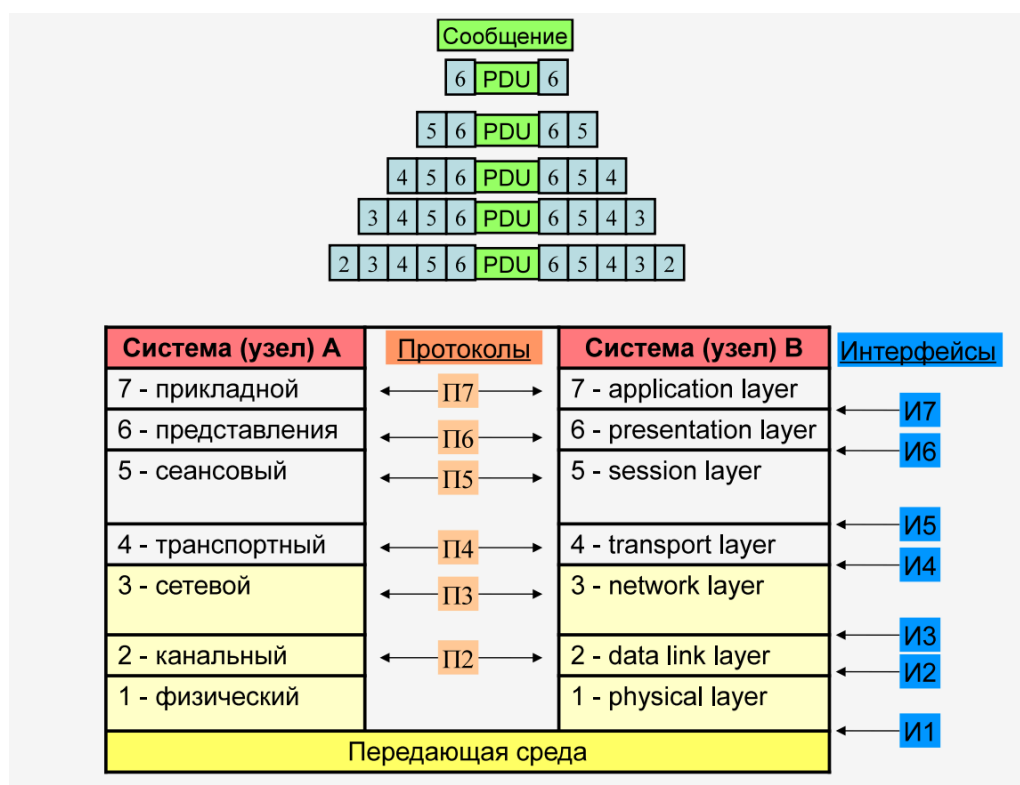
Иначе пиздеть будет только один.

Сообщение – законченное взаимодействие.

Есть заголовок и концевик.

Пакеты делают постоянной длины, а последний меньше будет.

71. Модель взаимодействия открытых систем (OSI)



Пришли к выводу, что существует 7-ми уровневая эталонная модель взаимодействия OSI (*The Open Systems Interconnection model*).

В реальной жизни прикладной, транспортный, сетевой, канальный, физический.

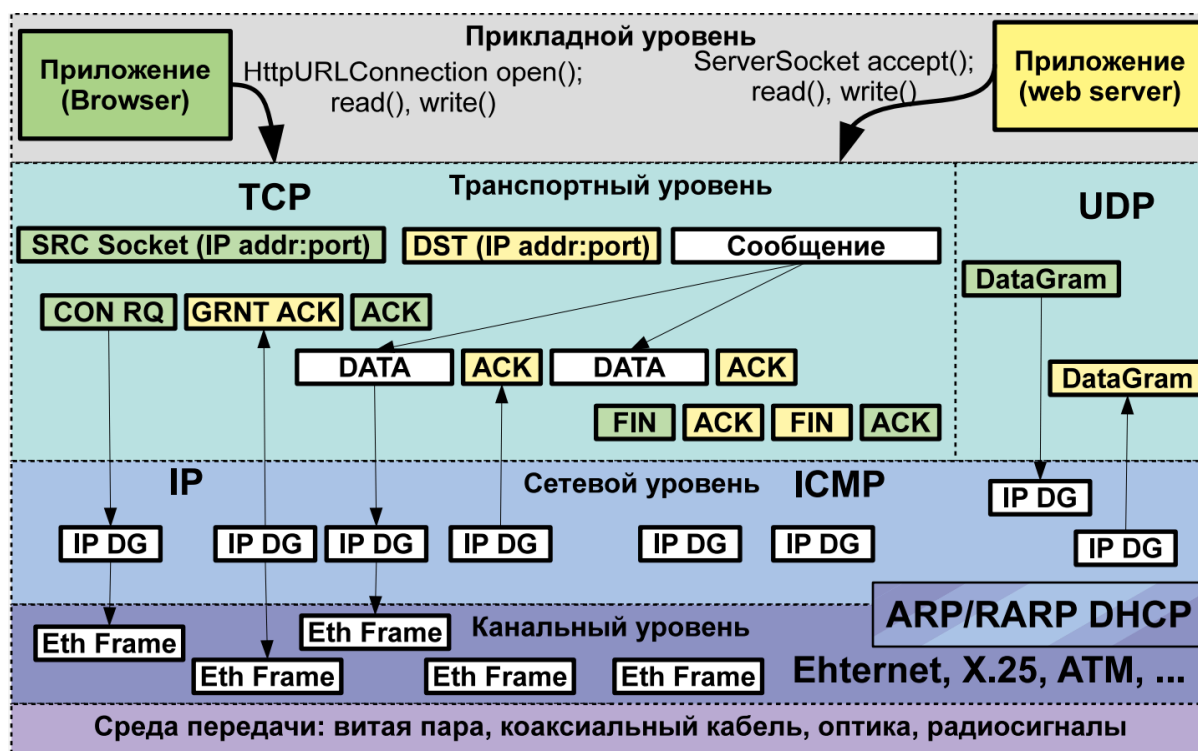
Каждый уровень оборачивает в свой заголовок.

Все сетевые сообщения вложены друг в друга.

Сеансовый уровень – коннект с удаленным узлом.

Сетевой уровень – информация между сегментами дальше

72. Модель TCP/IP



Желтое - приложение, и пакеты приходящие с сервера, Зеленые - браузер.

Прикладной уровень

Приложения, такие как браузеры или веб-серверы, используют этот уровень для взаимодействия с транспортным.

На прикладном уровне приложение работает с данными как с файлом — запрашивает ресурс и ожидает его получения.

Транспортный уровень

Дальше это попадает на транспортный уровень, где есть два параметра адресации.

Есть два протокола TCP — подтверждает доставку, UDP — простой и ненадежный, не проверяет дошло ли сообщение.

Точка взаимодействия состоит из IP адреса и порта.

Гнездо или сокет - соответствие между IP и портом.

Сокеты есть как у клиента, так и у сервера.

На уровне tcp наше сообщение бьется на пакеты данных.

Организуется виртуальный канал

Ждет разрешения подключения

Начинаем передавать данные

И получаем подтверждения, что данные пришли.

Транспортный уровень одевает ваше сообщение в заголовки. Добавляет сокет транспортного уровня - источник и куда надо отправить.

Сетевой уровень

Основная задача передавать пакеты между узлами сети.

Канальный уровень

На канальном уровне обрастает заголовками опять и тд. На каждом узле происходит развертывание и оборачивание в новые заголовки.

Канальный уровень нужен для передачи данных в рамках одной локальной сети по Ethernet.

ARP/RARP преобразуют IP в Ethernet адрес и наоборот.

Сервер посылает завершение соединения, юзер подтверждает.

73. Уровень передающей среды

- Коаксиальный кабель (устарел)
 - «толстый» - 10Base-5 — до 500м
 - «тонкий» - 10Base-2 — до 50м
- Витая пара 10Base-T, 100Base-T,
 - Категория 3: от 10 до 100 Мбит/с 100BASE-T4 (100м).
 - Категория 5е: 100 Мбит/с (2 пары), 1Гбит/с на (4пары)
 - Категория 6: 10 Гбит/с (55м)
 - Категория 7а: 40Гбит/с (50м), 100Гбит/с (15м)



Первый уровень – уровень передающей среды.

Сегмент сети – один провод, с ответвлениями к компам.

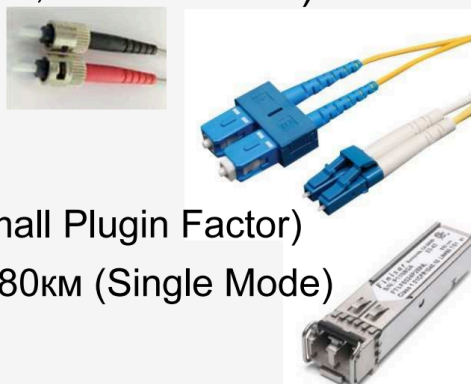
Используются 5 категория (стандартная) и 6, 7. С увеличением скорости передачи уменьшается расстояние.

Плюс витых пар: что если отъебывал провод, то страдает только тот, у кого отъебнуло (слова клименкова)

Витая пара уменьшает ошибки (в обе стороны идет сигнал, второй компенсирует помехи).

- Оптика (10BASE-F, 100BASE-SX, 10GBASE-ER...)

- ST (Straight Tip)
- SC (Standard Connector)
- LC (Lucent Connector)
- Лазер находится в SFP (Small Plugin Factor)
- ~500 м (Multi-mode fiber), ~80км (Single Mode)



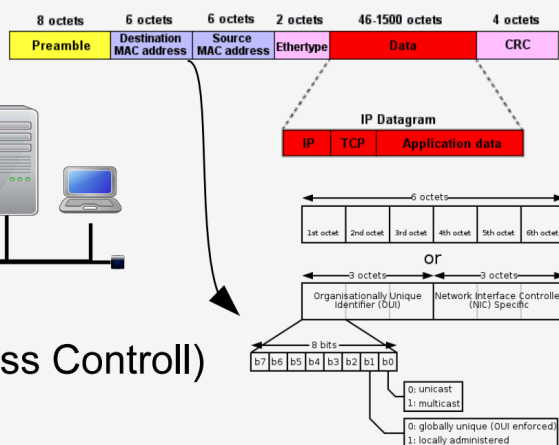
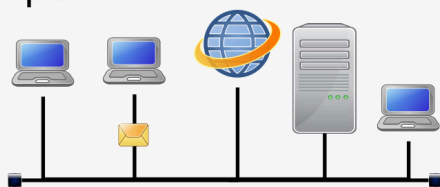
- Wireless (802.11 - WiFi, 802.16 - WiMAX, 3G, 4G)

- 2.4, 5, 60 GHz
- До 15 Гбит/с

Оптика – сингл мод и мульти мод (зависит от лучей лазера).
Используется для передачи на дальние расстояния.

74. Канальный уровень Ethernet

- Фреймы



- MAC (Media Access Control)

```
serge@helios:/home/serge> arp -a
```

Net to Media Table: IPv4

Device	IP Address	Mask	Flags	Phys Addr
igb0	192.168.10.211	255.255.255.255	o	00:0c:29:6c:7d:93
igb0	sunrise.cs.ifmo.ru	255.255.255.255	o	00:00:5e:00:01:0a
igb0	192.168.10.214	255.255.255.255	o	00:0c:29:99:13:e3
igb0	224.101.101.101	255.255.255.255		01:00:5e:65:65:65

Локальный сегмент (часть сети, где устройства могут общаться друг с другом напрямую, без участия маршрутизатора) характеризуется MAC адресами.

В отличие от IP-адреса, который может изменяться, MAC-адрес физически привязан к оборудованию и уникален.

При передаче данных по локальной сети каждый кадр Ethernet содержит:

- *Source MAC Address*: MAC-адрес устройства-отправителя.
- *Destination MAC Address*: MAC-адрес устройства-получателя.

Есть юникастовые (один комп с другим) есть мультикастовые пакеты (несколько компов выбирается), бродкаст (все адаптеры Ethernet).

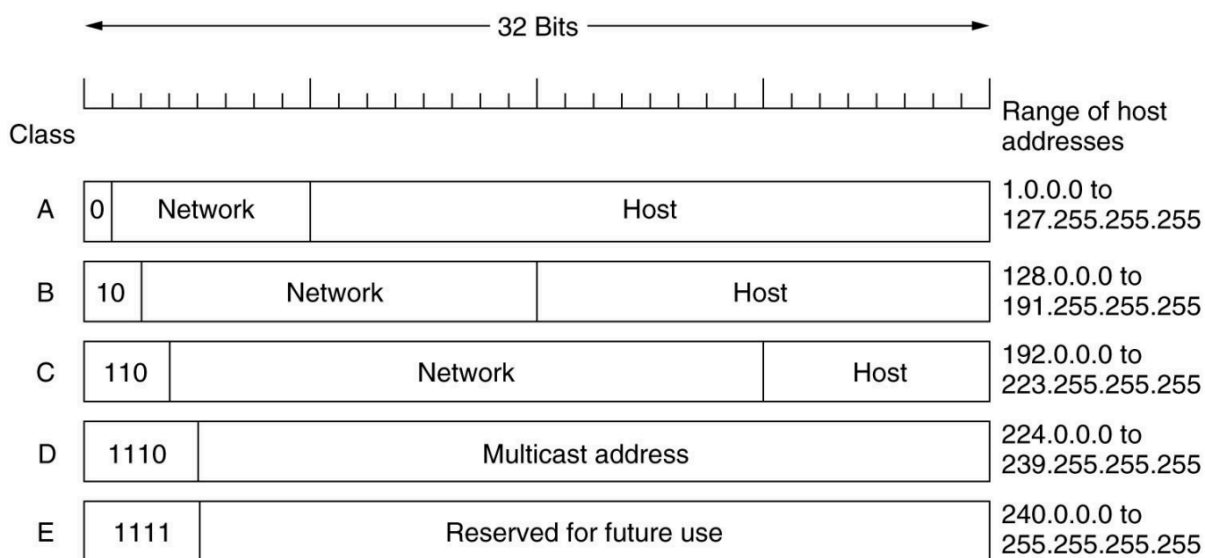
Есть таблица соответствия между IP и MAC. Используется пакет ARP, преобразовывающий IP адреса в MAC.

Бродкаст посылает IP на все компы в сети и получает в ответе от владельца данного IP его MAC адрес.

MAC добавляется в таблицу. ARP в следующий раз будет отправлять запрос на прямую по MAC адресу

Таблица локальна.

75. Сетевой уровень IP



Адрес делится на две части – сети и хоста.

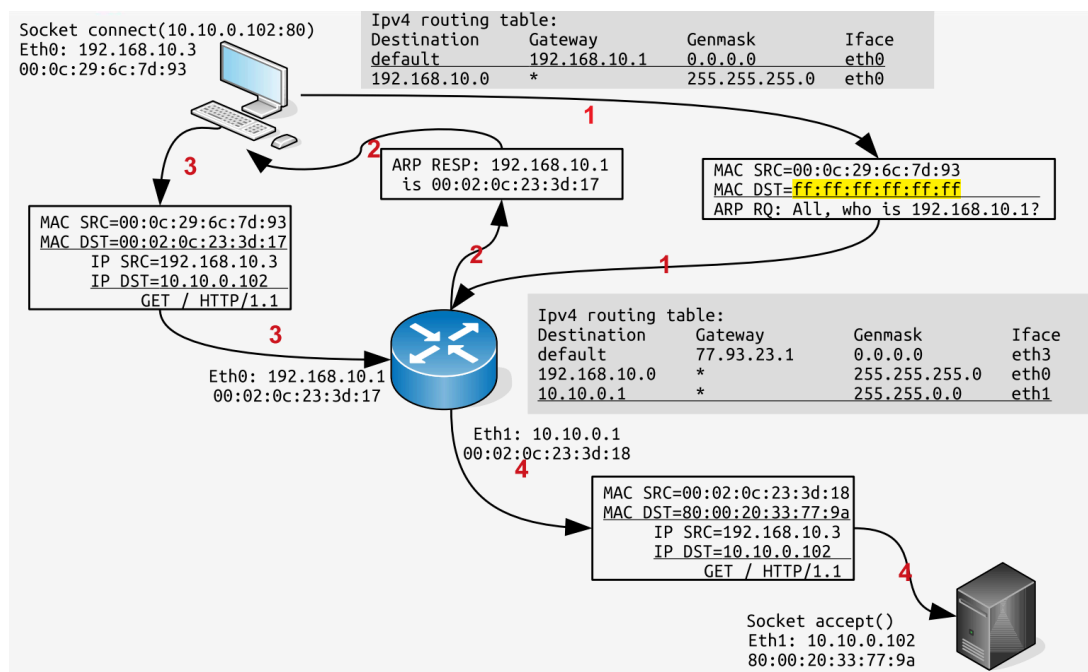
Если послать на последний IP, то посылается бродкаст на все компы в сети, кто первый ответит – тот принял.

Обычно используется уровень C.

Используется три параметра: свои MAC и IP, получателя и IP-адрес получателя (если устройство в той же сети) или IP-адрес маршрутизатора, если получатель в другой сети. В случае, если отправитель находится в другой подсети, используется дефолтный маршрутизатор для пересылки пакетов.

76. Сетевой уровень IP: маршрутизация

Нужно знать свой адрес, адрес назначения и таблицу.



Комп хочет послать веб запрос на сервер 10.10.0.102.

- 1) Смотрит в свою таблицу маршрутизации. Вторая строчка говорит, что сеть 192... – это локалка.
- 2) Отправляет по адресу по умолчанию.
- 3) Формируется ARP запрос. Посылает всей сети, ищет нужный адрес, получает MAC адрес, возвращает значение в таблицу.
- 4) Формируется запрос, пакет.
- 5) Пакет сравнивается с MAC адресом, отправляется на следующий уровень.
- 6) Если не туда пришел пакет, то уже там в таблице маршрутизации ищет нужный. Переправляет на нужный интерфейс.
- 7) Дальше также используется ARP. Формируется пакет, в котором изменены только MAC адреса, обернутые только что.
- 8) Сервер по MAC понимает, что пакет дошел до него и отправляет на прикладной уровень.

77. DHCP

Динамически выделяются и контролируются адреса. Отдает конфигурационную информацию.

78. Сервис имен DNS и другие

Надо давать IP адресам норм имена.
Простейший способ – /etc/hosts

DNS – глобально реализовано.

Иерархически построена. Есть корневые сервера в сети интернет, которые содержат зоны, которые содержат свои сервера, которые содержат инфу.

То есть в виде дерева.

Вопрос к днс происходит локально

Ищется по ns сверху (ru → ifmo → cs → helios). Запрашивает по зонам.

Чтобы хранить доп информацию, появились службы каталогов в BSD и OS Solaris (NIS, NIS+).

79. Транспортный уровень

- TCP — Transmission Control Protocol
 - Надежный, управляет перепосылкой данных
 - Организует виртуальное соединение между гнездами (Socket=IP:port) на двух системах
 - HTTP, FTP, SSH, SMTP, ...
- UDP — User Datagramm Protocol
 - Послал сообщение и забыл
 - Контроль надежности оставлен разработчику
 - Максимальная скорость передачи
 - SNMP, TFTP, DHCP, DNS, ...

TCP - нагружен, так как много всяких запросов высокая административная нагрузка

UDP - похуй на это

Смотришь видео, его корежит это потеря пакета из-за udp но быстрый кайф

80. Прикладной уровень

Протоколы разрабатывает программист.